

**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

CCDS25-1034: Full Stack Development and
Deployment of
a Web-Based Speech Recognition System

Final Report

Soh Zi Xiang
U2321798D

Project Supervisor: Assoc. Prof. Chng Eng Siong
Examiner: Asst. Prof. Zhang Mengmi

College of Computing and Data Science
Nanyang Technological University
Academic Year 2025/2026

NANYANG TECHNOLOGICAL UNIVERSITY

CCDS25-1034:

**Full Stack Development and Deployment of
a Web-Based Speech Recognition System**

Submitted in Partial Fulfillment of the Requirements
for the Degree of Bachelor of Engineering in Computer Science
of Nanyang Technological University

by

SOH ZI XIANG

College of Computing and Data Science
Academic Year 2025/2026

Abstract

MAGOR is a web-based speech recognition and transcription platform for processing audio and video recordings. Having been developed over multiple years by different developers, the platform had built up unstable components due to incomplete implementations, outdated dependencies, and broken service integrations, affecting its reliability and maintainability. This project addresses these issues through full-stack development and system enhancements, restoring broken administrative functionalities, implementing transcript and media download capabilities, and improving transcript editing workflows and user interface (UI) consistency. Artificial Intelligence (AI)-based transcript processing was also introduced through transcript summarization and question-and-answer (Q&A) features.

Acknowledgment

I express my sincere gratitude to Associate Professor Chng Eng Siong for providing me with the opportunity to work on the MAGOR platform as part of this Final Year Project. His guidance and feedback throughout the project have been invaluable in helping me bring it to completion.

I would also like to express my appreciation to my project mentor, Mr Kyaw Zin Tun, for his continuous guidance and support throughout the project. His patient guidance, timely feedback, and regular discussions have helped keep me on track and refine my work throughout the project.

My thanks go to my family, peers, and friends for their continuous support throughout my academic journey. Their encouragement, understanding, and companionship have kept me motivated and grounded to see this project through.

This research is supported by the RIE2025 Industry Alignment Fund – Industry Collaboration Projects (IAF-ICP) (Grant No. I2501E0041), administered by A*STAR, as well as supported by Schaeffler (Singapore) PTE. LTD. and NTU Singapore through Schaeffler-NTU Corporate Lab: Intelligent Mechatronics Hub.

Contents

1	Introduction	8
1.1	Background	8
1.2	Motivation	9
1.3	Objectives and Project Scope	10
1.4	Contributions Made	11
2	Literature Review	12
2.1	Related Works	12
2.2	Software Maintenance and Dependency Management	13
2.3	Relevant Natural Language Processing Research	13
2.3.1	Automatic Text Summarization	14
2.3.2	Hallucination in LLMs	14
3	MAGOR Architecture and Technologies	15
3.1	Overview	15
3.2	System Architecture	15
3.3	Technologies Used	18
3.3.1	Docker	18
3.3.2	MongoDB	19
3.3.3	Express.js	19
3.3.4	React.js	20
3.3.5	React Admin	20
3.3.6	SpeechLab Gateway	21
4	Solutions and Implementations	22
4.1	Admin Panel Restoration	23
4.1.1	User Management	23
4.1.2	Recording Management	25
4.1.3	ASR Request Management	26
4.1.4	Statistics Page	27

4.2	Transcript and Media Export	29
4.2.1	Multi-Format Transcript Conversion	29
4.2.2	Multi-File Export and ZIP Packaging	30
4.2.3	Media Download	31
4.3	Recording and Transcript Improvements	32
4.3.1	Subtitle Synchronization	32
4.3.2	Speaker Count Submission	34
4.3.3	Speaker Tag Editing and Duplicate-Name Handling	35
4.3.4	Transcript Studio Alignment	36
4.3.5	Transcript Selection and Editor Synchronization	37
4.3.6	Recordings Page UI Improvements	38
4.4	Storage and Deletion Consistency	42
4.4.1	Studio Deletion and Associated File Cleanup	42
4.4.2	Admin-Panel Recording Deletion Behavior	43
4.5	AI-Based Transcript Interaction	43
4.5.1	AI Summarization	43
4.5.2	Custom Summary Instructions	46
4.5.3	Q&A	48
4.6	Dependency and Infrastructure Upgrades	50
4.6.1	Node.js Runtime Upgrade	51
4.6.2	Dependency Upgrades	51
5	Conclusions and Future Works	53
5.1	Conclusion	53
5.2	Future Works	54

List of Figures

1.1	MAGOR recording playback page	9
3.1	MAGOR system architecture	16
3.2	MAGOR Studio architecture	17
3.3	Redis-backed queue implementation	18
3.4	MAGOR Docker container architecture	19
3.5	magor database collections in MongoDB Compass	20
3.6	Express.js route definitions in MAGOR Studio	21
4.1	Admin Panel Users list	23
4.2	Create User form with input validation	24
4.3	Deletion handling in dataProvider.js	25
4.4	Admin Panel Recordings list view	26
4.5	Admin Panel ASR Requests page	27
4.6	Admin Panel Statistics page	28
4.7	DownloadButton in Transcript Editor	29
4.8	Multi-format transcript download with ZIP export	31
4.9	Media playback controls and download option	31
4.10	Upload file preview	32
4.11	Normal caption display	33
4.12	Cluttered caption display in previous implementation	33
4.13	Updated transcription job request form	34
4.14	Transcription job request body	35
4.15	Duplicate speaker names with numeric suffixes	36
4.16	Non-functional “Transcript” drop-down in the Transcript Editor	37
4.17	Transcript selector in the Transcript Editor following the change	38
4.18	Recordings page with excessive metadata spacing	39
4.19	Expandable recording description	39
4.20	“Show options” panel opened via the dedicated button	40
4.21	Transcript auto-scrolling during playback	41
4.22	Restructured Recordings page layout	41

4.23 AI summarization process	44
4.24 Summarization request payload with the formatted transcript in the dialogue field	45
4.25 Manual summary polling interface	45
4.26 Generated transcript summary	46
4.27 Summarization request with default userPrompt	47
4.28 Custom summary input with user instructions	47
4.29 qas collection in MongoDB Compass	48
4.30 Q&A request payload showing the chatHistory message sequence	49
4.31 Q&A interface	50

List of Tables

4.1	Database model and field changes for Statistics	28
4.2	Docker base image changes for the Node.js 24 upgrade	51
4.3	Package changes across the three services	52

Chapter 1

Introduction

1.1 Background

MAGOR is a web-based audio and video transcription platform developed by the Speech and Language Laboratory at Nanyang Technological University (NTU), Singapore, led by Associate Professor Chng Eng Siong. The platform integrates the laboratory's in-house Automatic Speech Recognition (ASR) services to allow users to upload audio and video recordings and obtain automatically generated transcripts.

The generated transcripts can then be reviewed and edited alongside their corresponding recording through the media playback feature. The platform also provides additional functionalities for managing uploaded recordings and their associated transcripts, including creating, editing, searching and deleting content, as well as administrative functions for managing platform data. An example of the recording playback interface is shown in Figure 1.1.

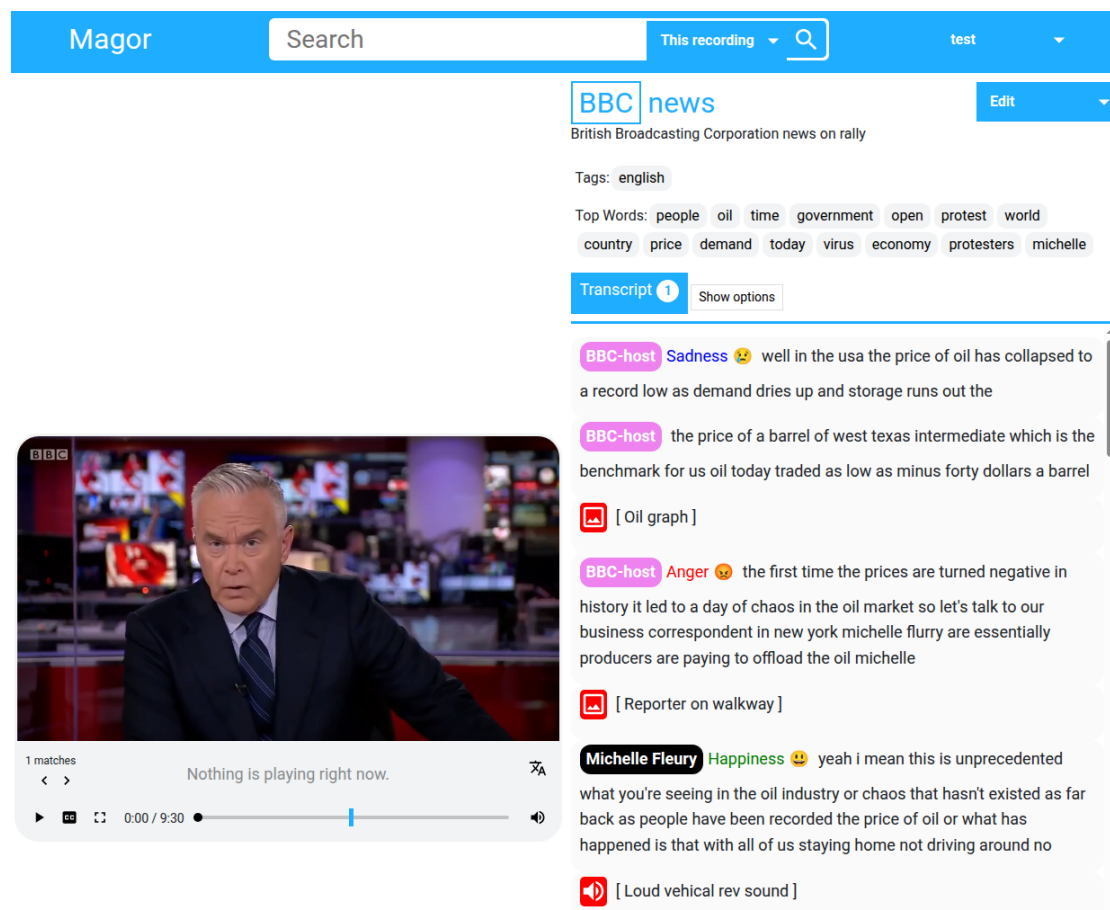


Figure 1.1: MAGOR recording playback page

1.2 Motivation

As MAGOR has been developed and maintained by different developers over the years, its code base has accumulated legacy code, outdated dependencies, partially completed implementations, and inconsistencies between system services.

During the initial system investigation, several major issues were identified across the platform. Multiple pages within the administrative panel had become partially non-functional due to broken API integrations. Certain transcript editing features were incomplete or unstable. Media playback pages contained subtitle synchronization and display issues. Download functionality for transcripts and recording files was either incomplete or entirely unavailable. Deleting a recording did not consistently remove its associated transcript and recording files, resulting in inconsistencies between the platform's database and its stored files. Additionally, several project dependencies and Node.js versions had become outdated over time, requiring updates to keep the platform aligned with current software versions.

Beyond restoring existing system functionality, there was also an opportunity to extend

the platform with Large Language Model (LLM)-based transcript processing capabilities. While MAGOR provides automatic speech recognition services for transcript generation, it did not previously support processing and interacting with transcript content using large language models. LLM-based processing could provide greater flexibility for users to more efficiently process and navigate transcripts, especially lengthy ones.

For instance, transcript summarization can condense the content into a more accessible format, enabling users to quickly extract essential insights without reading the entire transcript. Interactive question-and-answer capabilities, if implemented effectively, may improve engagement and allow users to ask questions directly about the content and obtain relevant answers without manually searching through or reviewing the recording and transcript.

This project was therefore undertaken to improve system stability, restore broken functionalities, improve code base maintainability, and add LLM-based features for transcript summarization and Q&A capabilities.

1.3 Objectives and Project Scope

Driven by these issues, the project focuses on two main goals: restoring and improving the reliability, maintainability and usability of the existing MAGOR platform, and extending it with AI-based transcript processing capabilities.

The specific objectives and scope of the project are as follows:

- Restore broken system functionalities across frontend and backend services.
- Improve maintainability by addressing legacy code, outdated references, and inconsistencies within the code base.
- Improve usability through interface improvements and user workflow enhancements.
- Improve system infrastructure through dependency upgrades and Docker configuration fixes.
- Enhance transcript processing capabilities with AI-based features.

The project focuses on maintaining and extending the existing MAGOR system rather than developing a completely new application. The development of the underlying ASR and LLM services is outside the scope of this project. Instead, the work focuses on integrating and using these services within MAGOR.

1.4 Contributions Made

This section provides an overview of the main contributions made over the course of the project. Details of the implementation and reasoning are discussed in subsequent chapters.

The main contributions are summarized as follows:

- Restored functionality of multiple Admin Panel pages including User Management, Recording Management, Statistics, and ASR Requests pages.
- Implemented transcript download support across multiple file formats, with ZIP export for multiple transcript downloads.
- Implemented media file download functionality to allow users to download their uploaded recordings.
- Implemented cascading deletion of recordings and their associated files and database records to keep stored files and database records consistent.
- Resolved subtitle rendering and playback synchronization issues.
- Added a speaker count field to allow users to specify the expected number of speakers when submitting transcription requests.
- Implemented speaker name modification in transcripts, including duplicate-name protection to maintain unique speaker labels.
- Aligned the Transcript Studio page with the Studio page across multiple areas, including transcript download support, transcript name editing, default selection, and sort order.
- Improved recording-page usability through interface enhancements, including description truncation and consolidating transcript controls, filtering options, and additional recording information into a single panel.
- Implemented AI-based transcript summarization and a Q&A feature within a shared “Ask AI” interface.
- Upgraded all MAGOR services to Node.js 24, resolving the dependency incompatibilities introduced by the upgrade.

Chapter 2

Literature Review

This chapter reviews related work previously performed on the MAGOR platform. Section 2.1 reviews the prior MAGOR projects that established the platform’s original architecture and functionality. Section 2.2 then reviews work on software maintenance and dependency management, which relates to the restoration work that forms the majority of this project, while Section 2.3 reviews the research areas underlying the AI-based features introduced.

2.1 Related Works

Previous work on MAGOR spans several Final Year Projects and internal development cycles, focused primarily on transcript editing, deployment architecture, cloud integration, Docker containerization, and infrastructure standardization.

Huang’s earlier Final Year Project [1] implemented the platform’s transcript editor, allowing automatically generated transcripts to be reviewed and corrected alongside their corresponding recording. That report also documents the platform’s service architecture, including the separation between MAGOR Frontend, MAGOR Backend, and MAGOR Studio, and is the source of the architecture diagrams reproduced in Chapter 3.

Meanwhile, Yong’s subsequent Final Year Project [2] focused heavily on deployment optimization, removal of Azure cloud storage dependencies, software testing frameworks, Continuous Integration and Continuous Deployment (CI/CD) pipelines, and deployment standardization under multiple operating environments.

These earlier works established the core architecture, functionalities, and deployment infrastructure that the current MAGOR platform is built on. As the platform subsequently evolved, changes to its code base resulted in functional regressions and incomplete features documented in Chapter 4, which accumulated across the development cycles.

2.2 Software Maintenance and Dependency Management

Much of the work carried out in this project involved restoring and maintaining an existing system rather than developing new functionality, which relates to established research on software maintenance and the long-term degradation of software systems.

Software that is modified repeatedly over a long period tends to degrade, and the consequences extend beyond the structure of the code itself. In a systematic mapping study of 73 publications on architecture erosion, Li et al. [3] found that such degradation can manifest as violations of a system’s intended structure, reduced software quality and increased difficulty in modifying the system. It is this increased difficulty that was encountered throughout this project. Multiple modifications made by different developers across several development cycles had left behind disabled routes, outdated model and field references, and partially completed features. Resolving these issues required tracing their dependencies across the affected services before the original functionality could be restored or corrected, as documented in Chapter 4.

A related aspect of software maintenance is dependency management, particularly to keep dependencies updated to address known vulnerabilities and maintain compatibility with the runtime environment. In a study of 6,546 Node.js applications, Alfadel et al. [4] found that where an application was exposed to a publicly known vulnerability in one of its dependencies, a fix was already available but had not been applied by the application’s maintainers in 90.8% of cases, and that affected applications remained exposed for a considerable period, reported in the study as 103 days. Their findings identified the absence of dependency updates, rather than the absence of available fixes, as the main reason known vulnerabilities persist in deployed applications. This type of issue was also encountered in MAGOR at the start of this project, where all three services were running Node.js releases that no longer received security updates, and where several packages could not be installed on a current runtime. Thus, the dependency upgrades described in Section 4.6 involved not only updating package versions but also resolving compatibility issues introduced by upgrading the Node.js runtime.

2.3 Relevant Natural Language Processing Research

Beyond prior MAGOR developments, the AI-based features introduced in this project draw inspiration from established research areas in natural language processing. This section reviews relevant literature on text summarization and on hallucination in language model output, and relates each to a corresponding design decision made in the implementation elaborated in Chapter 4 of the report.

2.3.1 Automatic Text Summarization

Automatic summarization is a long-established task in natural language processing. Rao et al. [5] surveyed 226 studies on abstractive summarization published between 2011 and 2023, in which a summary is newly composed rather than pieced together from sentences copied out of the source, and examined the challenges, datasets, and evaluation methods used in the field.

The summaries produced by the feature described in Section 4.5.1 are abstractive in this sense, as the overview and key points returned by the language model are newly written rather than extracted verbatim from the transcript. The input, however, differs from the written documents typically considered in this literature. A MAGOR transcript is usually composed of a sequence of short spoken utterances containing filler words, repetitions, and incomplete sentences rather than well-formed prose. Given the nature of this input, the default prompt described in Section 4.5.2 instructs the model to disregard filler words and repetitions and to summarize the key ideas, opinions, decisions, and conclusions of the recording rather than its individual utterances.

2.3.2 Hallucination in LLMs

A recognized limitation of LLMs is their tendency to produce plausible but unsupported content, commonly referred to as hallucination. Huang et al. [6] surveyed this phenomenon and distinguished between factuality hallucination, where generated content conflicts with verifiable facts, and faithfulness hallucination, in which generated content does not accurately reflect the input provided to the model.

Faithfulness is particularly relevant to MAGOR because the generated summaries and answers describe the contents of one specific recording. The output should therefore remain consistent with the transcript provided to the model. The generated content can also include timestamps that the user can click to seek the player to the corresponding position (Section 4.5.1). A fabricated timestamp could therefore direct the user to a point in the recording that does not support the accompanying statement.

In a transcript-based application like MAGOR, prompt design can be used to constrain generated content to the source material. This is reflected in MAGOR’s implementation, where the summarization prompt explicitly instructs the model not to invent timestamps or information (Section 4.5.2), while Q&A requests instead supply the transcript through the SpeechLab gateway’s chat task, which applies its own system prompt rather than one defined in MAGOR (Section 4.5.3).

Chapter 3

MAGOR Architecture and Technologies

This chapter describes the overall architecture of the MAGOR platform and introduces the core technologies used throughout the system. As this project focuses on improving an existing production system rather than developing a new application from scratch, understanding the system architecture is vital for identifying dependencies between services and assessing how implemented changes affect the overall platform.

The chapter begins with an overview of MAGOR’s distributed service architecture (Section 3.1) and a description of how its services interact (Section 3.2), before introducing the core technologies used to build, deploy, and persist data for the platform (Section 3.3).

3.1 Overview

MAGOR follows a distributed full-stack architecture consisting of various interconnected services that collectively support authentication, media processing, transcription generation, transcript editing, database management, and external service communication. The platform consists of three primary services, referred to as MAGOR Frontend, MAGOR Backend, and MAGOR Studio. These services are coordinated through an Nginx reverse proxy and share a MongoDB database for persistent data storage.

3.2 System Architecture

The general service topology of MAGOR is illustrated in Figure 3.1, originally produced by Huang [1] during an earlier stage of MAGOR development. The diagram provides an overview of the core service architecture, including the relationships between MAGOR

Frontend, MAGOR Backend, MAGOR Studio, MongoDB and the ASR and image captioning services.

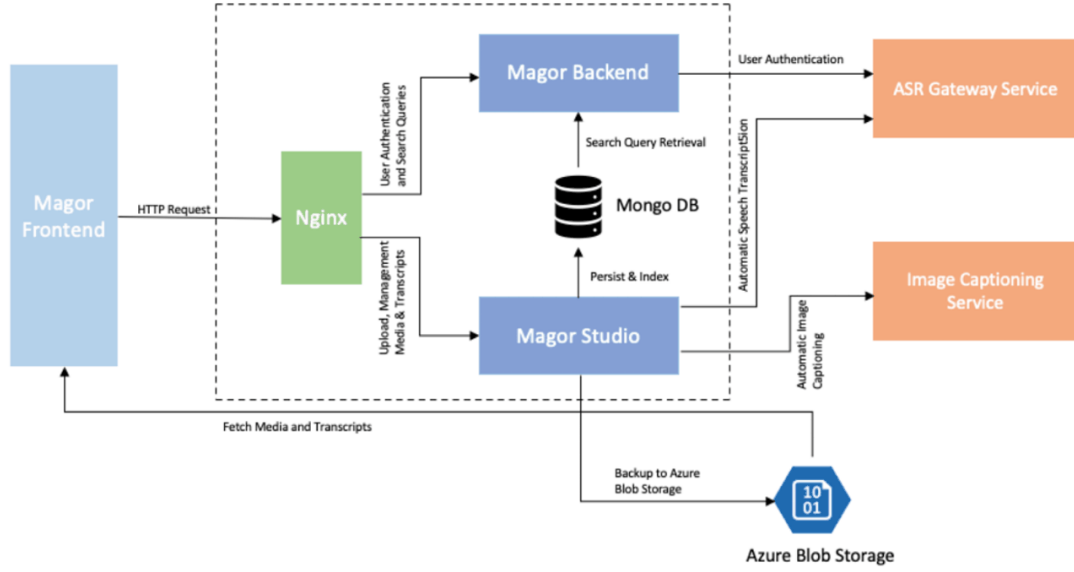


Figure 3.1: MAGOR system architecture (adapted from Huang [1])

Nginx is a web server that is frequently deployed as a reverse proxy, receiving client requests on a single entry point and forwarding them to the appropriate backend service [7]. The flow of requests begins from MAGOR Frontend and is routed through an Nginx reverse proxy, which forwards incoming client requests to either MAGOR Backend or MAGOR Studio depending on the URL path. Requests under `api` are directed to MAGOR Backend, while requests under `studioapi` are directed to MAGOR Studio.

MAGOR Frontend acts as the primary user interface through which users access and interact with the platform. Built with React.js, the application provides interfaces for the core functionality of the platform, including user login, recording management, transcription, transcript editing, and search. Requests generated by the frontend are sent through Nginx, which routes them to MAGOR Backend or MAGOR Studio for processing according to the requested functionality.

MAGOR Backend serves as the server-side component responsible for managing the core platform operations. This includes handling user authentication, administrative functionality, user management, and search queries. For authentication, the backend does not verify user credentials locally. Instead, login and registration requests are sent to the SpeechLab gateway described in Section 3.3.6, which verifies the credentials and issues an access token that the backend subsequently uses to identify the user when processing requests.

MAGOR Studio is the secondary backend server responsible for managing the transcrip-

tion workflow, which is handled separately from MAGOR Backend. This includes media uploads, transcript generation requests, transcript editing and export, and communication with external processing services. MAGOR Backend and MAGOR Studio share a single MongoDB database for persisting user accounts, recording metadata, transcripts, and administrative records. Both services access the shared database to retrieve and update the data required for their respective functions.

Figure 3.1 also depicts recordings and transcript files being backed up to Azure Blob Storage. However, this is not currently active. Although the corresponding Azure Blob Storage functionality remains present in the code base, uploaded recordings, transcripts, and generated assets are currently stored locally through a persistent Docker volume shared by MAGOR Backend and MAGOR Studio.

Redis is an in-memory data store that is used for purposes including caching, message brokering, and queueing [8]. MAGOR Studio also uses Redis to manage an asynchronous queue for search-index updates. A more detailed view of MAGOR Studio’s internal processing pipeline is shown in Figure 3.2, also originally produced by Huang [1]. It depicts incoming transcription and image-captioning requests being placed onto separate Redis-backed queues, picked up by background workers that forward each request to the corresponding external service, with a webhook listener receiving the response and queuing the result for transcript indexing before it is persisted to MongoDB. This diagram also depicts an Azure Blob Storage backup step that is not currently active.

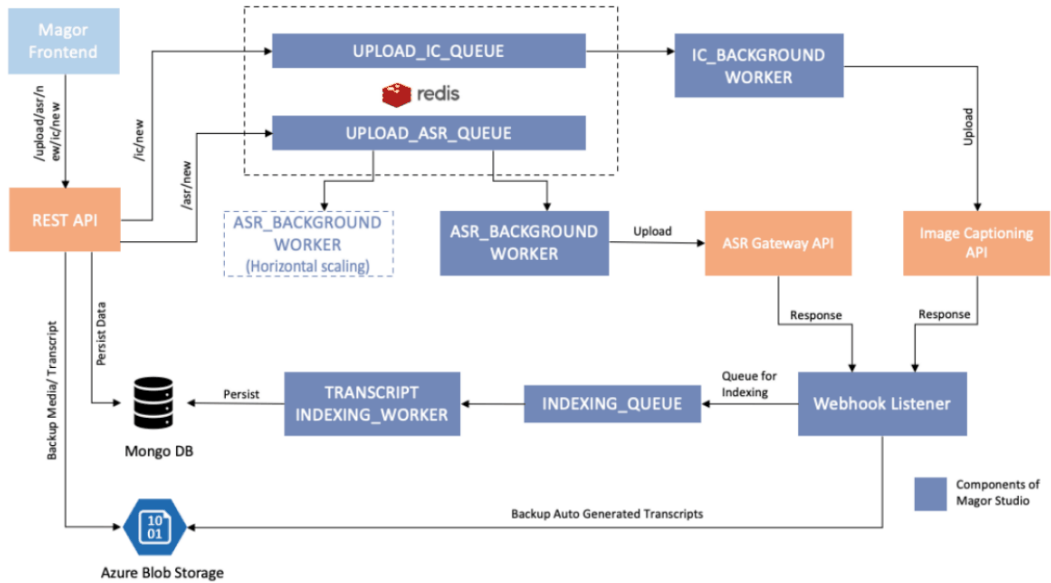


Figure 3.2: MAGOR Studio architecture (adapted from Huang [1])

Placing search-index updates on a Redis-backed queue allows MAGOR to continue pro-

cessing incoming requests without waiting for a recording’s search index to be rebuilt. If an indexing job fails, it is automatically retried rather than being dropped, with a fixed number of attempts and a delay between retries. Figure 3.3 shows the relevant implementation in `index.ts` of the transcript-indexer service, where the queue connects to Redis and enqueues newly processed recordings for re-indexing.

```

93 export var addMediaToIndexQueue;
94 if (process.env.NODE_ENV !== 'test') {
95   logger.info("redis index queue started");
96   const indexQueue = process.env.NODE_ENV !== 'test'
97     ? new Queue("index", { redis: { port: 6379, host: process.env.REDIS_HOSTNAME } }) : new Queue("index");
98   const CONCURRENT_UPLOAD_PROCESSES = 3;
99
100 > indexQueue.process(CONCURRENT_UPLOAD_PROCESSES, async (job, done) => { ...
109 > });
110 > indexQueue.on("completed", async (job, result) => { ...
113 > });
114 > indexQueue.on("failed", (job, error) => { ...
117 > });
118
119 addMediaToIndexQueue = (mediaId: string) => {
120   indexQueue.add(
121     { id: mediaId }, { attempts: 5, backoff: 5000 });
122 };
123 };
124

```

Figure 3.3: Redis-backed queue implementation

3.3 Technologies Used

3.3.1 Docker

Docker is a platform for developing and running applications in containers, packaging applications and their dependencies into isolated environments. Docker Compose extends this approach to multi-container applications by allowing services, networks, and storage volumes to be defined and managed through a single configuration file [9].

MAGOR is deployed using Docker Compose, with its main services including an Nginx reverse proxy, MongoDB, MAGOR Backend, MAGOR Frontend, and MAGOR Studio. These services are connected through a dedicated bridge network. Application files are stored on a named Docker Volume shared between MAGOR Backend and MAGOR Studio.

Figure 3.4, adapted from Yong [2], illustrates this container topology: client requests reach the reverse proxy first, which forwards them to the appropriate MAGOR service, while the Backend, Studio, and Frontend containers each connect to the shared MongoDB and Redis containers as needed. A separate seed container populates the database with an initial admin account before exiting.

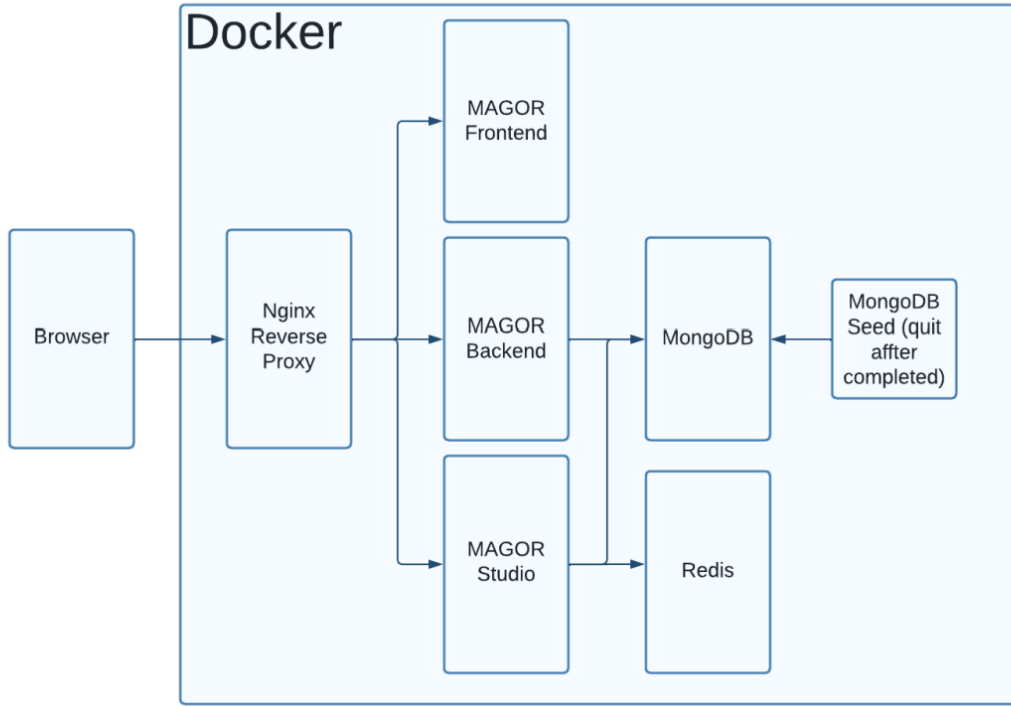


Figure 3.4: MAGOR Docker container architecture (adapted from Yong [2])

3.3.2 MongoDB

MongoDB is a document-oriented NoSQL database that stores records as documents rather than as rows in fixed relational tables, allowing the fields present to vary between documents within the same collection [10]. It is the primary database for the MAGOR platform, accessed through Mongoose by both MAGOR Backend and MAGOR Studio. The project’s collections include user accounts, recording metadata, transcripts, transcription (ASR) requests, and the Q&A records introduced for the Ask AI feature.

MongoDB Compass is a graphical interface for querying and inspecting MongoDB databases. It provides tools for querying collections, adding and modifying documents, deleting data, and exporting collection data, among other database management operations. MongoDB Compass was used throughout this project to view collection documents directly, verify schema and field-naming consistency between Backend and Studio, and diagnose data-related issues encountered while restoring platform functionality. An example of the `magor` database and its available collections is shown in Figure 3.5.

3.3.3 Express.js

Express.js is a minimal web application framework for Node.js that provides the routing and middleware used to handle incoming HTTP requests [11]. Both MAGOR Backend

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
asrrequests	-	12.29 kB	0 B	0	0 B	1	12.29 kB
icerequests	-	36.86 kB	202.00 B	2	101.00 B	1	36.86 kB
media	-	57.34 kB	49.18 kB	9	5.46 kB	2	110.59 kB
qas	-	40.96 kB	8.63 kB	16	539.00 B	1	36.86 kB
recordings	-	4.10 kB	0 B	0	0 B	2	8.19 kB
summaries	-	4.10 kB	0 B	0	0 B	3	12.29 kB
transcriptionrequests	-	32.77 kB	1.91 kB	8	239.00 B	1	32.77 kB
transcripts	-	36.86 kB	11.26 kB	23	489.00 B	1	32.77 kB
useraccesses	-	32.77 kB	9.39 kB	38	247.00 B	1	32.77 kB
users	-	36.86 kB	1.67 kB	5	334.00 B	2	65.54 kB

Figure 3.5: **magor** database collections in MongoDB Compass

and MAGOR Studio are Node.js applications built using Express.js. MAGOR Backend exposes routes for authentication, user and recording administration, and search, while MAGOR Studio exposes routes for media upload, transcript processing and export, and the AI-based transcript features. The two services expose their routes independently as REST APIs, which MAGOR Frontend calls to access the platform’s functionality. Figure 3.6 shows an example of these routes, defined in `routes.ts` in the upload routes of MAGOR Studio.

3.3.4 React.js

React.js is a widely popular JavaScript library for building user interfaces from reusable, stateful components [12]. MAGOR Frontend is implemented using React.js and forms the entire client-side application through which users interact with the platform. Shared components, including the transcript editor, navigation menus, drop-down menus, and modal dialogs, are reused across multiple pages within the application.

3.3.5 React Admin

The MAGOR Admin Panel is built using React Admin, a framework for constructing data-driven administration interfaces from a declarative set of resources [13]. The Admin Panel currently exposes four resources: **users**, **recordings**, **TranscriptionRequests** (labeled ASR Requests in the interface), and a custom statistics view. A shared data provider connects the resources to their corresponding backend API endpoint, translating React Admin’s list, edit, and delete operations into the appropriate HTTP requests.

```

1  import { Router } from "express";
2  import * as controller from './controller';
3  import * as validator from './validator';
4  import * as httpUtils from '../utils/http';
5
6  const router = Router();
7
8  router.post(
9    '/recording',
10   validator.validateUploadRecording,
11   httpUtils.handleValidationError,
12   controller.handleUploadRecording
13 );
14
15 router.post(
16   '/transcript',
17   validator.validateUploadTranscript,
18   httpUtils.handleValidationError,
19   controller.handleUploadTranscript
20 );
21
22 // PUT router for save draft transcript
23 router.put(
24   '/transcript',
25   controller.handleUpdateTranscript
26 );
27
28 export default router;

```

Figure 3.6: Express.js route definitions in MAGOR Studio

3.3.6 SpeechLab Gateway

MAGOR depends on an external service, referred to as the SpeechLab gateway and shown as the ASR service in Figure 3.1. The platform relies on it for three functions: verifying user credentials and issuing the access tokens used for authentication, performing ASR on uploaded recordings, and, following the features introduced in this project, processing LLM requests through its llm endpoint [14].

Chapter 4

Solutions and Implementations

This chapter describes the technical work and implementations carried out throughout the project, covering the maintenance, restoration and extension of the existing MAGOR platform. The work is organized into six main areas.

The restoration of the Admin Panel is presented in Section 4.1, addressing issues affecting the previously non-functional User Management, Recording Management, ASR Request Management, and Statistics pages, primarily caused by mismatched or outdated fields and disabled or incomplete backend routes.

Section 4.2 covers the transcript and media export functionality. This section introduces multi-format transcript downloads with ZIP packaging, together with the ability to download originally uploaded recording files.

Improvements to recording playback and transcript editing are discussed in Section 4.3, including subtitle synchronization, speaker count configuration for transcription job requests, alignment of the Studio and Transcript Studio pages, and improvements to the layout of the Recordings page.

Storage and deletion consistency is addressed in Section 4.4, where inconsistencies between database records and stored files during recording deletion are resolved through cascading cleanup of associated files and database records.

Section 4.5 presents the implementation of AI-based transcript interactions through the “Ask AI” feature, covering summarization and Q&A capabilities using the SpeechLab gateway’s LLM.

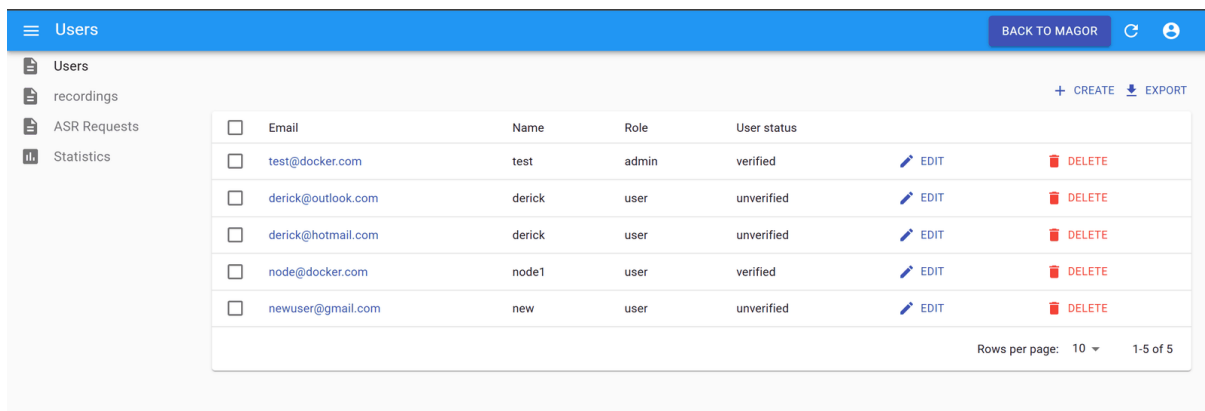
Finally, Section 4.6 documents the runtime and dependency upgrades applied across the three services, including the migration to Node.js 24, the Docker base image variants used for each service, and the package upgrades required to keep the platform building and running on the updated runtime.

4.1 Admin Panel Restoration

The MAGOR Admin Panel, built on React Admin, provides administrative interfaces for managing users, recordings, ASR requests, and platform statistics. At the beginning of this project, several of these interfaces revealed non-functional or unreliable operations across the MAGOR Frontend, MAGOR Backend API, and underlying MongoDB collections. Restoring each page required tracing the affected operations through these layers to identify the source of the failure before implementing a fix.

4.1.1 User Management

The Admin Panel's User Management page allows administrators to view, create, edit, and delete user accounts, but several of these operations did not function correctly. The users list, shown in Figure 4.1, provides an overview of the user accounts managed through the Admin Panel.



☐	Email	Name	Role	User status		
☐	test@docker.com	test	admin	verified	EDIT	DELETE
☐	derick@outlook.com	derick	user	unverified	EDIT	DELETE
☐	derick@hotmail.com	derick	user	unverified	EDIT	DELETE
☐	node@docker.com	node1	user	verified	EDIT	DELETE
☐	newuser@gmail.com	new	user	unverified	EDIT	DELETE

Figure 4.1: Admin Panel Users list

Creating a new user through the Admin Panel could report a failure even when the user had, in fact, been created. The issue was caused by the MAGOR Backend response not including the newly created user's identifier in the form expected by the Admin Panel. As part of restoring this workflow, explicit email and password format validation was added to the user creation form (`Create.js`), so that malformed input is rejected before a request is submitted, together with clearer success and failure notifications for administrators, as shown in Figure 4.2.

The screenshot shows a 'Create User' form. The header is blue with a hamburger menu, the text 'Create User', and a 'BACK TO MAGOR' button. The sidebar on the left lists 'Users', 'recordings', 'asrRequests', and 'Statistics'. The form fields are: 'Name' with the value 'derick'; 'Email' with the value 'derick@hotmail' and a red error message 'Email must be in the format name@example.com'; 'Password' with a red error message 'Password must be at least 6 characters' and a toggle icon; and 'Role'. A 'SAVE' button is at the bottom.

Figure 4.2: Create User form with input validation

A related issue affected the user status field. Setting a user’s status to “unverified” through the Edit page failed with an unprocessable-entity error. The cause was a mismatch between the identifier the Admin Panel submitted for this status and the identifier the MAGOR Backend’s user schema expected. The User model defines the value as “notVerified” (`user.js`), while the Admin Panel previously submitted the value as “unverified”. This was corrected in `Edit.js`, where the status field now submits `notVerified` as the underlying identifier while displaying “Unverified” on the user interface.

Deleting multiple users simultaneously also failed. React Admin’s bulk-deletion action calls the data provider’s `deleteMany()` function with a filter describing the selected records, but the MAGOR Backend only exposes a single-item deletion route (`DELETE /users/:id`). No route accepted a filtered bulk-delete request. Rather than introduce a new backend bulk-deletion endpoint, the `deleteMany()` function in `dataProvider.js` was implemented to iterate over the selected user identifiers and issue an individual deletion request for each one, reusing the existing single-item deletion route and its associated authorization and validation logic to implement bulk user deletion. Figure 4.3 shows the `delete()` and `deleteMany()` functions in `dataProvider.js`, illustrating how bulk deletion reuses the same single-item deletion request issued by `delete()`.

```

320     delete: (resource, params) =>
321         resource === 'recordings'
322         ? deleteRecordingById(params.id)
323         : httpClient(`${apiUrl}/${resource}/${params.id}`, {
324             method: 'DELETE',
325         }).then(({ json }) => ({ data: json })),
326
327     deleteMany: (resource, params) => {
328         if (resource === 'recordings') {
329             return Promise.all(
330                 params.ids.map((id) => deleteRecordingById(id))
331             ).then(() => ({ data: params.ids }))
332         }
333
334         return Promise.all(
335             params.ids.map((id) =>
336                 httpClient(`${apiUrl}/${resource}/${id}`, {
337                     method: 'DELETE',
338                 })
339             )
340         ).then(() => ({ data: params.ids }))
341     },
342

```

Figure 4.3: Deletion handling in `dataProvider.js`

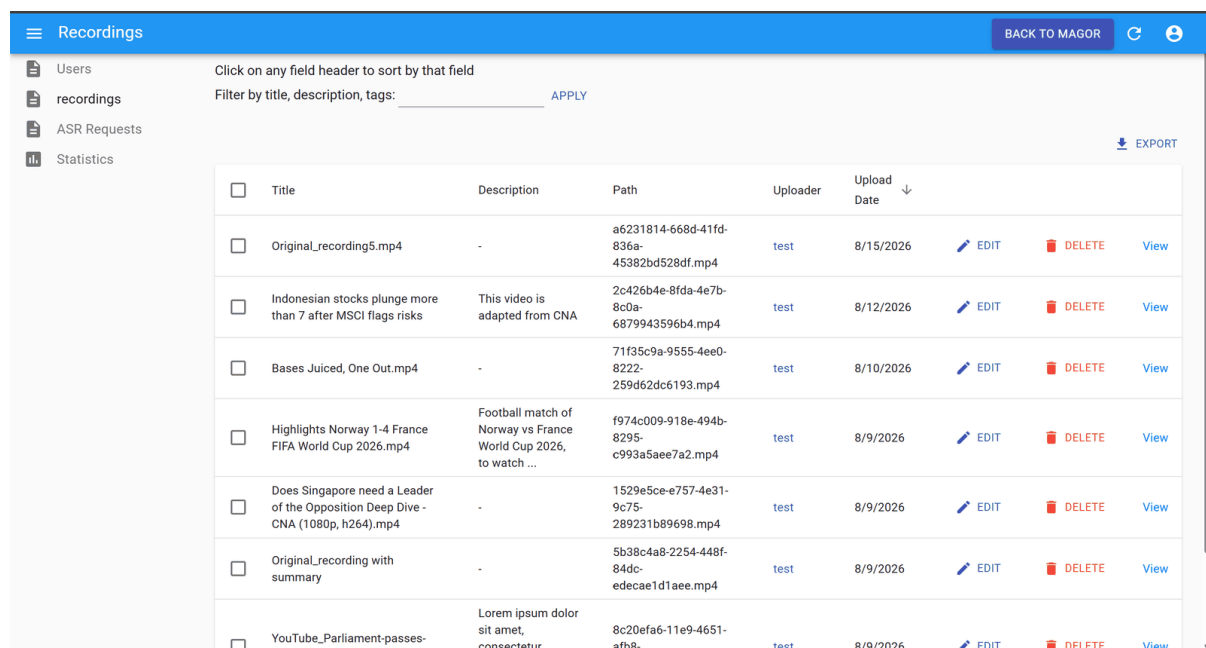
4.1.2 Recording Management

The Admin Panel’s Recordings page allows administrators to browse, edit, and delete uploaded recordings, but several of its fields and actions did not function correctly.

First, the recording list failed to display or sort by upload date. This was caused by a mismatch between the field name the Admin Panel expected (“`uploadDate`”) and the field MAGOR Backend’s Media model actually populates (“`createdAt`”). Correcting the mismatch allowed the upload dates to be retrieved correctly by the Admin Panel. The list view was then updated in the Recording directory’s `List.js` to use `createdAt` for both displaying and sorting the upload date.

The recording’s storage path also failed to render reliably. Because recordings may reference their file location under different fields depending on how and when they were uploaded, the path column now falls back across a sequence of potential candidate fields (`tempResourceUrl`, `azureResourceUrl`, `path`, `url`) rather than reading a single fixed field. Of these, only `tempResourceUrl` is used in the current upload flow, while the remaining fields are retained for recordings created through earlier versions of the system. To prevent long paths from causing the table to overflow, the displayed value is truncated to its filename component, with the full path displayed in a hover tooltip. These changes

are reflected in the Recordings list view, as shown in Figure 4.4.



☐	Title	Description	Path	Uploader	Upload Date	
☐	Original_recording5.mp4	-	a6231814-668d-41fd-836a-45382bd528df.mp4	test	8/15/2026	EDIT DELETE View
☐	Indonesian stocks plunge more than 7 after MSCI flags risks	This video is adapted from CNA	2c426b4e-8fda-4e7b-8c0a-6879943596b4.mp4	test	8/12/2026	EDIT DELETE View
☐	Bases Juiced, One Out.mp4	-	71f35c9a-9555-4ee0-8222-259d62dc6193.mp4	test	8/10/2026	EDIT DELETE View
☐	Highlights Norway 1-4 France FIFA World Cup 2026.mp4	Football match of Norway vs France World Cup 2026, to watch ...	f974c009-918e-494b-8295-c993a5aee7a2.mp4	test	8/9/2026	EDIT DELETE View
☐	Does Singapore need a Leader of the Opposition Deep Dive - CNA (1080p, h264).mp4	-	1529e5ce-e757-4e31-9c75-289231b89698.mp4	test	8/9/2026	EDIT DELETE View
☐	Original_recording with summary	-	5b38c4a8-2254-448f-84dc-edecae1d1aee.mp4	test	8/9/2026	EDIT DELETE View
☐	YouTube_Parliament-passes-	Lorem ipsum dolor sit amet, consectetur	8c20efa6-11e9-4651-afb8-	test	8/9/2026	EDIT DELETE View

Figure 4.4: Admin Panel Recordings list view

Beyond the data display issues, editing and deleting recordings from the Admin Panel also previously failed because the corresponding backend routes were disabled. The update and delete routes in `recordings.js` in the routes folder were then re-enabled to call `controller.updateItem()` and `controller.deleteItem()` directly against the Media model.

For recording creation, however, the ability to create a new recording directly from the Admin Panel was removed. This is because recording creation is handled mainly through MAGOR Studio's upload workflow, while the Admin Panel focuses on the administrative operations provided by MAGOR Backend. As such, the corresponding backend route and the `create` action for recordings in the Admin Panel are disabled in the current implementation. This avoids having multiple paths for creating recordings and keeps recording processing within MAGOR Studio's workflow.

4.1.3 ASR Request Management

The Admin Panel's ASR Requests page is used to monitor and manage automatic speech recognition jobs submitted through MAGOR Studio. This page did not load any requests, and deleting a request did not function correctly.

Initially, the page's failure to load any requests was caused by the corresponding backend route handling having been disabled. Once re-enabled, the page still returned no results because it queried an outdated `asrRequest` collection rather than the corresponding

`TranscriptionRequest` collection that MAGOR Studio currently writes ASR job records to. After these issues were corrected, the route was updated to retrieve items from the `TranscriptionRequest` collection and normalize each record's identifier field before returning it to the Admin Panel, allowing the list to be rendered correctly.

Similar to the previous Admin Panel pages, deletion functionality for ASR Requests was restored. Single deletion is handled through `deleteItem()`, while a `deleteMany()` handler was also added to support bulk deletion of `TranscriptionRequest` documents from the database.

The status of each ASR request has also been updated with color-coded labels, with green indicating successful requests, yellow indicating pending, and red indicating failed requests. This allows administrators to quickly distinguish between request statuses at a glance, as shown in Figure 4.5.

☐	Recording ID	Recording	ASR ID	Status	Date ↓	
☐	6a1530601048bd0023103aa5		6a15306663309cdb335d43e0	Success	5/26/2026	View on Magor
☐	6a152d6714f3860023a87170	Bases Juiced, One Out.mp4	6a152d7363309cdb335d4226	Pending	5/26/2026	View on Magor
☐	69b2730fdb3f0002339be00	YouTube_Parliament-passes-motion-considering-Pri_Media_67m11XNfN3w_001_1080p	69b27321b356c587a85aa827	Success	3/12/2026	View on Magor
☐	69b0b7d6f7044e0023c57d96	Indonesian stocks plunge more than 7 after MSCI flags risks - CNA (144p, h264, youtube).mp4	69b0bb1eb356c587a859c8a4	Success	3/11/2026	View on Magor
☐	69afce860852900242ca7c6	Does Singapore need a Leader of the Opposition Deep Dive - CNA (1080p, h264).mp4	69afcd4ab356c587a8592411	Success	3/10/2026	View on Magor

Rows per page: 10 1-5 of 5

Figure 4.5: Admin Panel ASR Requests page

4.1.4 Statistics Page

The Admin Panel's Statistics page aggregates platform-wide metrics on recordings and ASR requests. It previously failed to load because errors in the underlying aggregation queries caused the page to become stuck during loading.

The core issue was that the page's queries were referencing database models and fields that no longer matched the platform's current schema. Investigating the failure revealed that the `Recordings` and `asrRequests` models referenced in the aggregation logic had since been renamed to `Media` and `TranscriptionRequests`, and several duration and caption-related fields had changed name or structure, as summarized in Table 4.1. This was corrected by updating the aggregation queries to reference the `Media` and `TranscriptionRequests` models and their corresponding field identifiers, with con-

ditional handling added where required to accommodate legacy field names. Default values are also returned when an aggregation produces no results to prevent the page from failing when no matching data exists.

Type	Previous Identifier	Current Identifier
Model	Recordings	Media
Model	asrRequests	TranscriptionRequests
Field	durationInSeconds	duration
Field	transcripts	_transcripts
Field	imageCaptions	_imageCaptions
Field	soundCaptions	_soundCaptions

Table 4.1: Database model and field changes for Statistics

Figure 4.6 shows the Statistics page, which presents its statistics in two main sections: Recordings and ASR Requests. The Recordings section reports the total number of recordings, along with the average, minimum, and maximum values for duration, tag count, unique word count, and transcript count, as well as the most frequently used tags. The ASR Requests section reports the total number of requests, the number of completed requests, and the oldest incomplete request, allowing administrators to identify requests that have remained unresolved for the longest time.

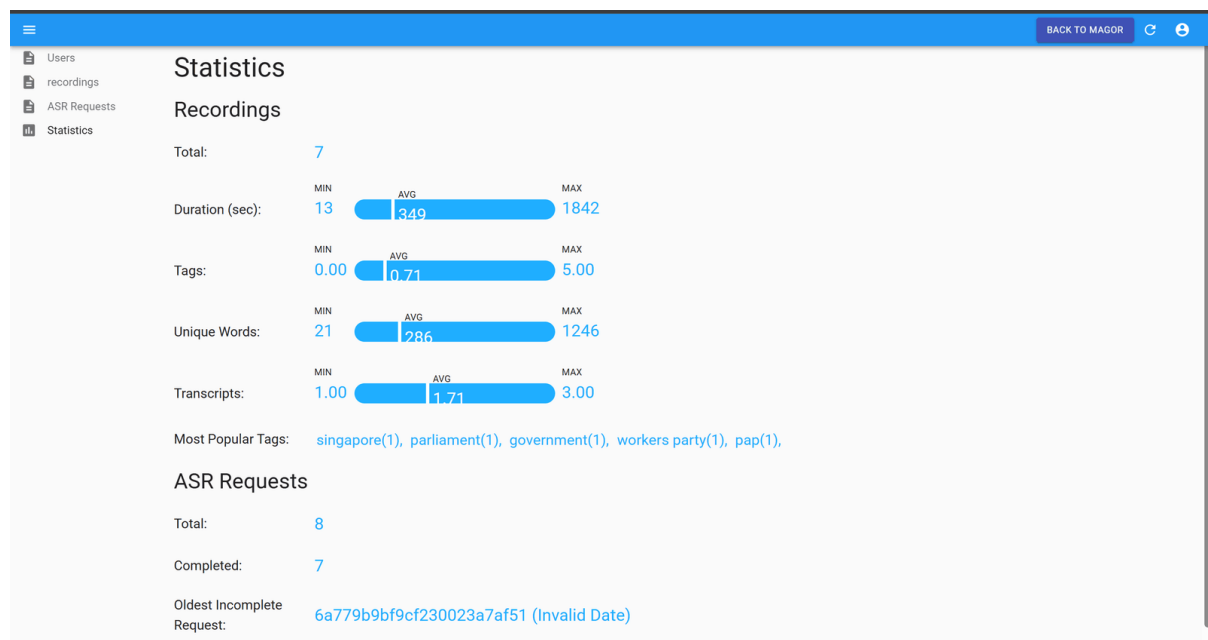


Figure 4.6: Admin Panel Statistics page

4.2 Transcript and Media Export

MAGOR Studio manages transcript generation and storage. As part of this project, functionality was added to allow generated transcripts to be exported in multiple file formats and for uploaded recordings to be downloaded through the Studio interface.

4.2.1 Multi-Format Transcript Conversion

Previously, MAGOR Studio provided a download option for generated transcripts. However, it was limited to the transcript's original XML format. The download control button was refactored into a reusable `DownloadButton` component, providing a selectable list of output formats that includes the original XML and the newly added STM, TXT, SRT, DOCX, and TextGrid formats. Figure 4.7 shows the resulting format selection within the Transcript Editor page.

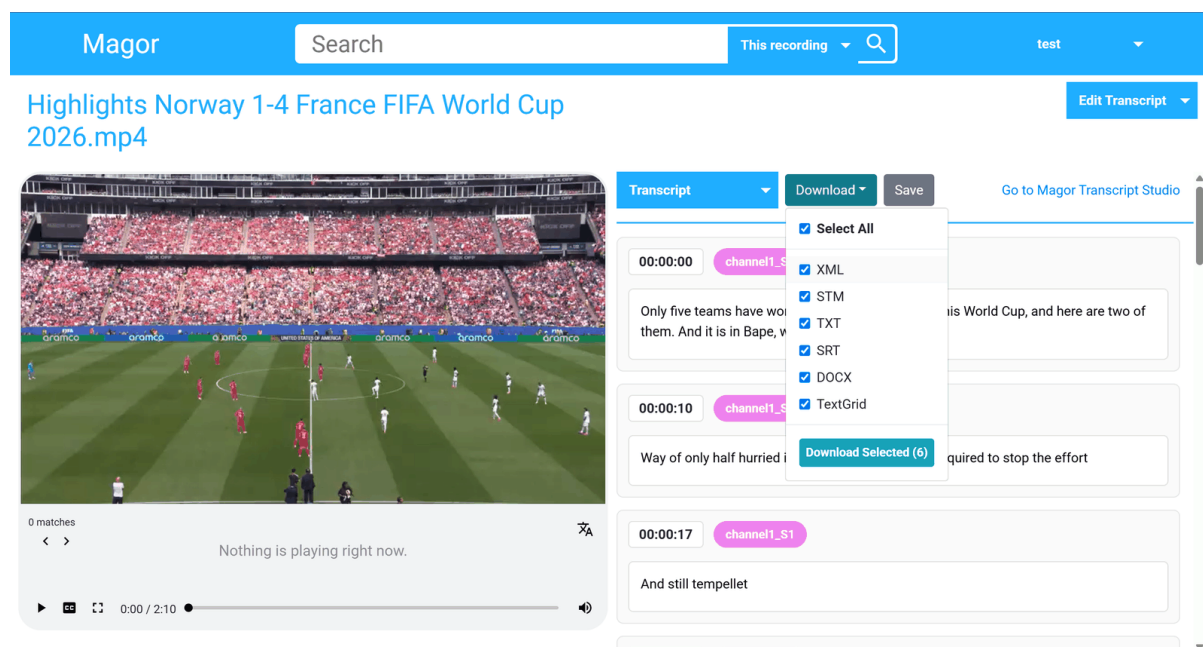


Figure 4.7: DownloadButton in Transcript Editor

The conversion process follows a sequence: the original XML transcript file is parsed into a JavaScript object, normalized into a common internal representation consisting of per-segment speaker identifiers, start and end times, transcript text, and word-level timing, and then converted into the requested output format from this normalized representation.

Each format applies its own formatting rules:

- **STM**: Produces one line per speech segment with speaker and timing information.
- **TXT**: Produces a timestamped, human-readable transcript.

- **SRT**: Produces sequentially numbered subtitle entries with timestamps formatted in HH:MM:SS,mmm and the speaker identifier included in each entry.
- **DOCX**: Produces a Word document with each speaker’s text rendered in a distinct color.
- **TextGrid**: Produces a Praat-compatible interval tier per speaker.

4.2.2 Multi-File Export and ZIP Packaging

In the initial implementation, the transcript download control allowed only one output format to be downloaded at a time. When multiple formats were requested together, each was triggered as a separate browser download using staggered `setTimeout()` calls, resulting in multiple separate downloaded files rather than a single consolidated export.

The previous download approach was replaced with server-side ZIP packaging. The download endpoint in `controller.ts` in the transcript routes accepts a comma-separated list of requested formats. When more than one format is requested, each format is converted using the process described in Section 4.2.1, and the resulting files are packaged into a single ZIP archive using the `JSZip` library before being returned to the client in a single response. A request for only a single file format continues to return that format directly, without packaging.

MAGOR Frontend’s `DownloadButton` component presents the available formats as a checklist, including a “Select All” option, and issues a request listing every selected format. This removed the need for the earlier staggered client-side download logic, as selected formats are now submitted in a single request, as shown in Figure 4.8.

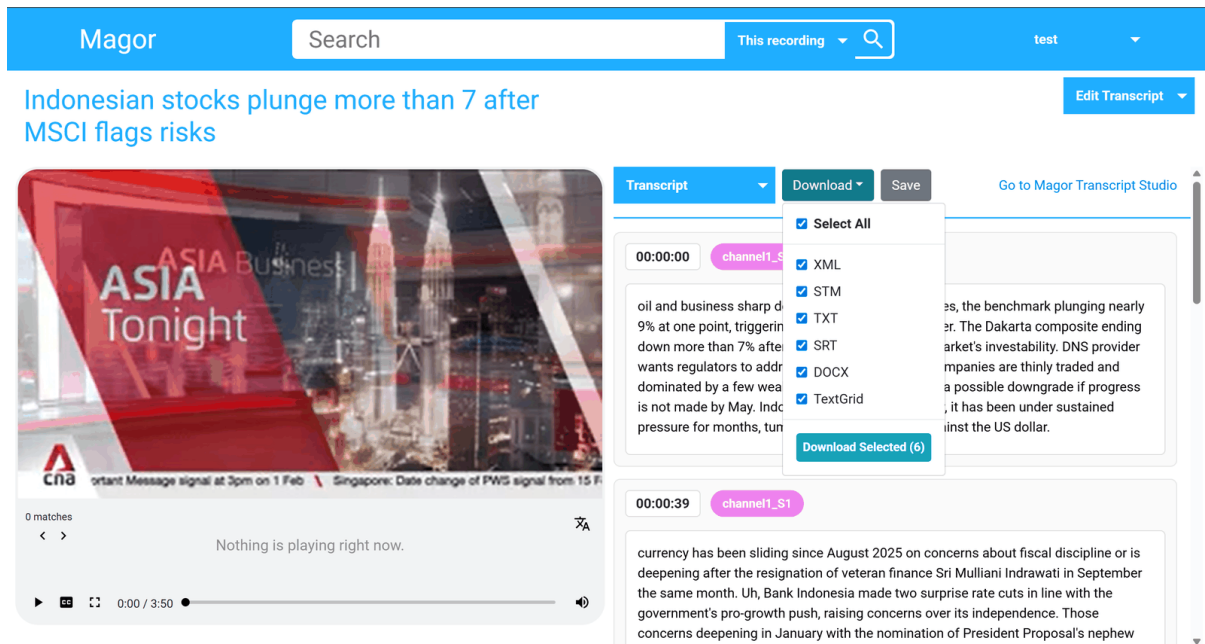


Figure 4.8: Multi-format transcript download with ZIP export

4.2.3 Media Download

Prior to this project, users could upload audio and video recordings through MAGOR Studio but had no way to retrieve the originally uploaded file afterwards. A download control was added to the existing `VideoPreview` and `AudioPreview` components. Each component now accepts an optional `downloadUrl` prop. When provided, a download icon linking directly to the recording file is rendered alongside the existing playback controls, as shown in Figure 4.9.

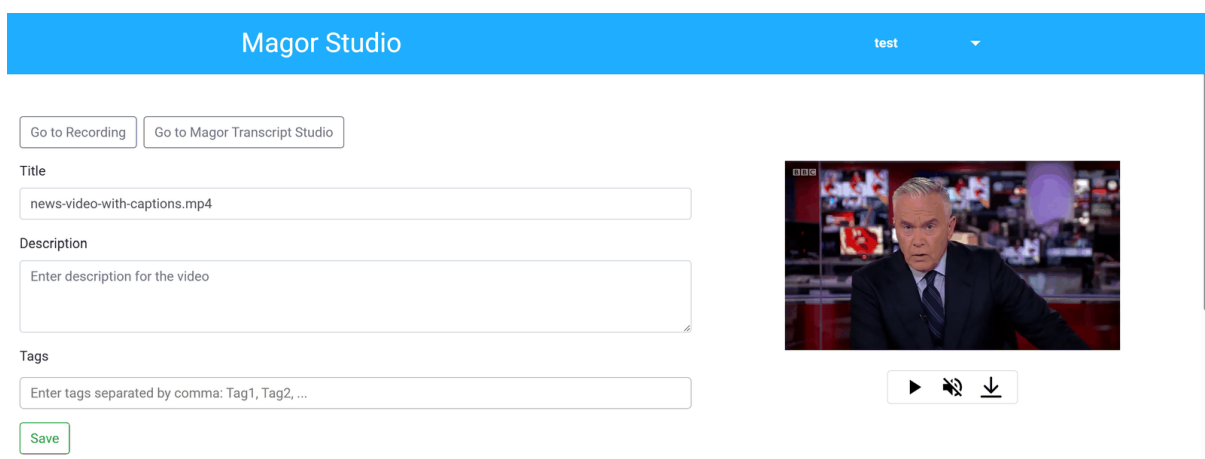


Figure 4.9: Media playback controls and download option

The same `VideoPreview` and `AudioPreview` components are also used by the upload modal to preview a file before it is uploaded, as shown in Figure 4.10. Since the recording

file has not yet been uploaded to the server, no `downloadUrl` can be supplied, and no download icon is rendered, preventing download requests from being initiated before a server-side file path is available.

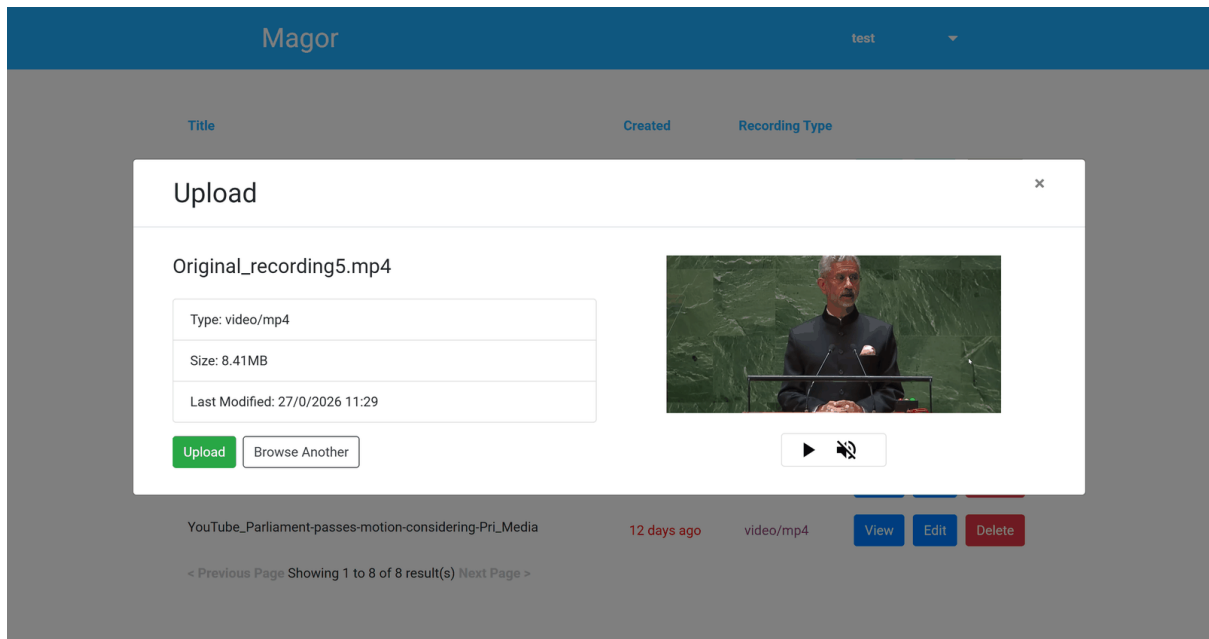


Figure 4.10: Upload file preview

With this implementation, users can download the original audio or video file directly from the playback controls only after the recording has been successfully uploaded.

4.3 Recording and Transcript Improvements

MAGOR's recording playback page and transcript editing interface are major components for reviewing and correcting generated transcripts. This section describes the UI and functional improvements made across the Recordings, Studio, and Transcript Studio pages.

4.3.1 Subtitle Synchronization

MAGOR's recording playback page displays captions beneath the video or audio player that follow the transcript during playback. The caption display shows three captions at a time: the previous caption, the current caption, and the next upcoming caption. The current caption is displayed most prominently, while the previous and next captions are displayed with reduced opacity, allowing users to distinguish the current spoken text while retaining the surrounding transcript context, as shown in Figure 4.11.

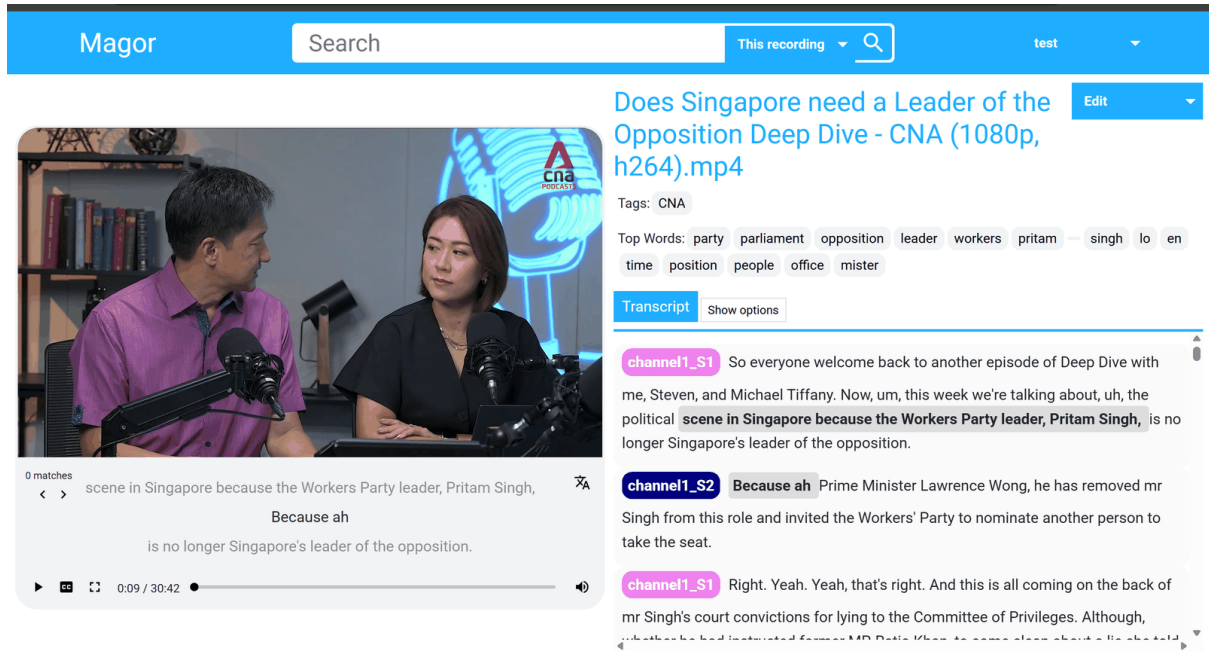


Figure 4.11: Normal caption display

In the previous implementation, although it functioned correctly in most cases, the current caption could occasionally contain multiple sentences across several lines, rather than maintaining the intended display of previous, current and next captions. This caused the caption block to extend beyond its intended layout, resulting in a cluttered caption display, as shown in Figure 4.12.

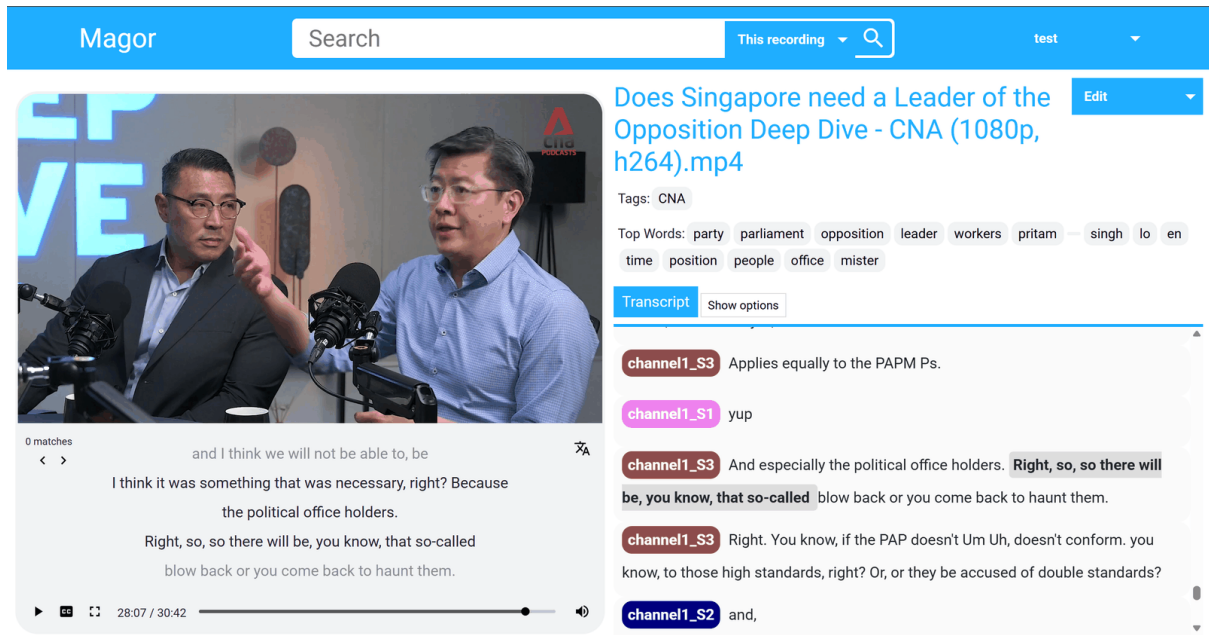


Figure 4.12: Cluttered caption display in previous implementation

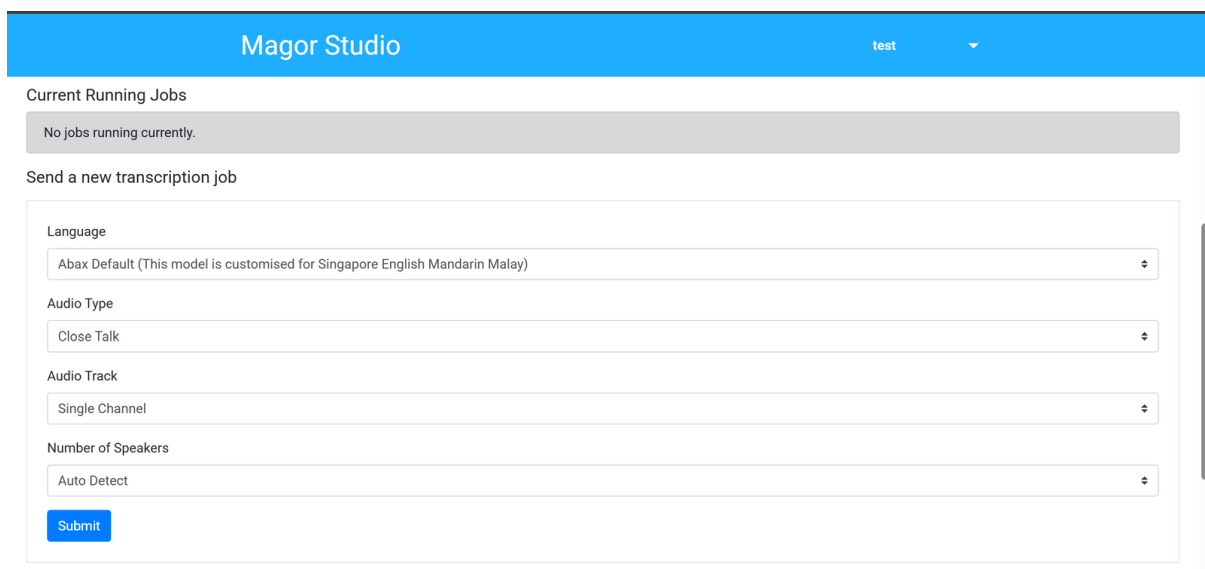
This occurred because the previous implementation determined the current caption using

`timeline.seekToAndGetDiff(seek)`, which reported captions that had started or ended since the previous playback position check. This check was performed each time the player reported a new playback position. When more than one caption started between two consecutive checks, such as during a seek or when captions were closely spaced in time, more than one caption could be retained in the display at once.

To address this issue, the caption displayed as the current caption was changed to be determined directly from the current playback position. The speech captions are stored in an array and sorted by their start time, after which the active caption is selected when the current playback position is greater than or equal to its start time and less than its end time (`startTime <= seek && seek < endTime`). If multiple captions overlap at the same position, the last matching caption in the array is selected.

4.3.2 Speaker Count Submission

The Studio page on the MAGOR platform provides a form for submitting a recording for ASR transcription. To allow the speaker count used during transcription to be specified more precisely, a “Number of Speakers” control was added to this form, allowing users to specify the expected number of speakers in the recording, rather than relying solely on the automatic speaker count detection performed by the SpeechLab gateway. The updated form is shown in Figure 4.13.



The screenshot shows the 'Magor Studio' interface. At the top, there's a blue header with 'Magor Studio' and a 'test' dropdown. Below the header, a section titled 'Current Running Jobs' shows 'No jobs running currently.' in a grey box. The main section is 'Send a new transcription job', which contains a form with four dropdown menus: 'Language' (set to 'Abax Default (This model is customised for Singapore English Mandarin Malay)'), 'Audio Type' (set to 'Close Talk'), 'Audio Track' (set to 'Single Channel'), and 'Number of Speakers' (set to 'Auto Detect'). A blue 'Submit' button is at the bottom of the form.

Figure 4.13: Updated transcription job request form

The “Number of Speakers” input presents values from one to five as quick-select options, with “Auto Detect” used when no value is specified, though a value can also be typed in directly. On submission, `index.js` in the Studio folder maps this value to a `customData: { numSpk }` object, using `-1` when no value is selected, and includes it in the ASR job

submission request. Before the request is sent to the SpeechLab gateway, the value is converted to the flattened field name (`customData[numSpk]`) expected by the gateway, matching the format documented by the gateway's API [14], as shown in Figure 4.14.

```
2026-06-04T09:59:52.444Z - info: ASR form body preview:
{
  "lang": "trilingual",
  "audioType": "closetalk",
  "audioTrack": "single",
  "customData[numSpk]": 9,
  "webhook": "http://20.205.178.30/studioapi/asr/webhook",
  "outputFormats": [
    "xml"
  ]
} [/usr/src/app/dist/services/api-asr-gateway/index.js]
```

Figure 4.14: Transcription job request body

4.3.3 Speaker Tag Editing and Duplicate-Name Handling

Transcript segments generated by automatic speaker diarization are labeled with speaker identifiers by default, such as `channel1_S1`, which are not necessarily meaningful to the end users. To make these identifiers easier to interpret, the ability to rename speakers directly within the transcript display and transcript editor was added when viewing the transcript in the Recordings page or when viewing the Transcript Editor page. In both interfaces, a speaker name can be clicked and edited directly, after which the user can enter a new name and save the change.

When a speaker is renamed, the new name is checked against the other speaker identifiers in the transcript. If the requested name conflicts with an existing identifier belonging to a different speaker, a numeric suffix is automatically appended and incremented until a unique name is obtained. For example, renaming a speaker to **Charlie** when another speaker is already named **Charlie** results in **Charlie_2**, preventing two distinct speakers from being displayed under the same name, as shown in Figure 4.15. The renamed identifier is then updated on the transcript's underlying XML file and saved as a draft, which is subsequently reloaded to reflect the updated name change.

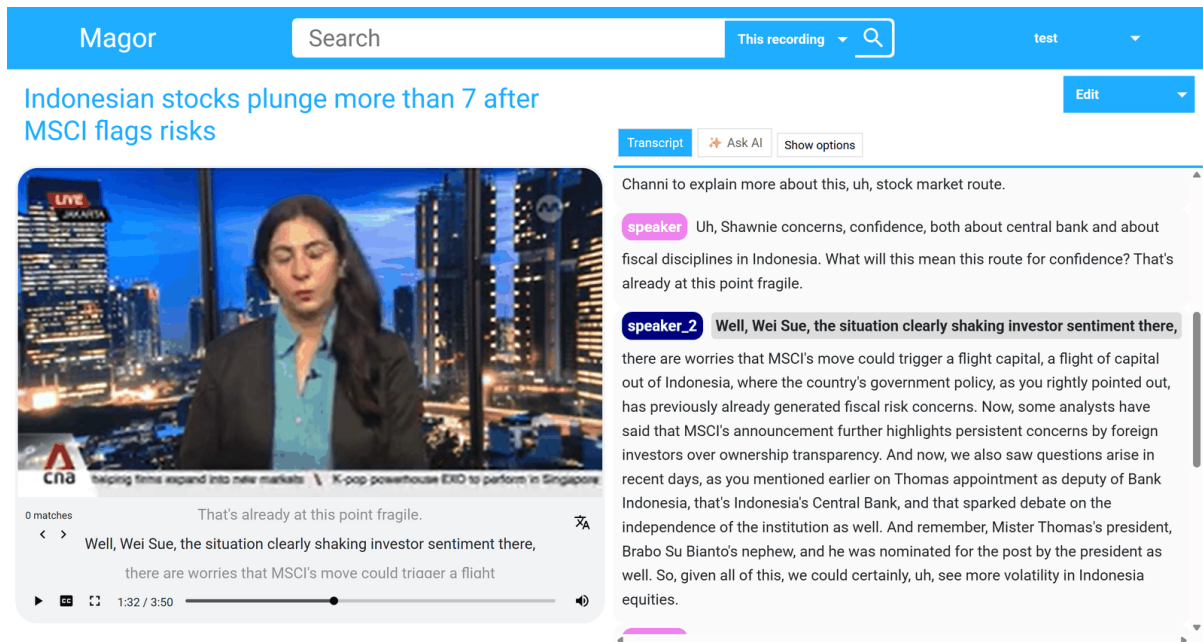


Figure 4.15: Duplicate speaker names with numeric suffixes

4.3.4 Transcript Studio Alignment

MAGOR Studio displays a list of a recording's transcripts at the bottom of the Studio page, and the Transcript Studio page provides a similar list through a separate interface. Several changes brought the Transcript Studio page's list into closer alignment with the Studio page's, as differences in the available functions between two similar lists could lead to inconsistent user expectations when navigating between interfaces. The changes are summarized as follows:

- **Transcript Renaming:** Transcript renaming was already available on the Studio page, but was not available on the Transcript Studio page. As part of this project, the same name field and Save control were added to the Transcript Studio page, allowing users to rename transcripts directly from the transcript list.
- **Transcript Download:** Transcript download support was extended to the Transcript Studio page through the existing `DownloadButton` component described in Section 4.2.2, which is shared and reused directly between the Studio and Transcript Studio pages.
- **Set Default Control:** A "Set Default" control, allowing a transcript to be marked as the default for its recording, was already implemented in the Studio page. However, this option was not available in the Transcript Studio page. To ensure consistent workflows on both sides, the "Set Default" control was added to the Transcript Studio page.

- **Default Ordering:** The Transcript Studio page’s default transcript ordering was also changed from chronological order to reverse chronological order, with the most recently created transcripts displayed first. This change was applied to both the initial list load and the default selection in the sort drop-down, matching the ordering used on the Studio page.

4.3.5 Transcript Selection and Editor Synchronization

A recording may have more than one transcript associated with it, and both the Recordings page and the Transcript Editor page allow a specific transcript to be selected for viewing. The Transcript Editor page previously displayed a “Transcript” drop-down containing “Change Transcript” and “Clear Transcript” options, but the control was not connected to any functionality, so selecting either option had no effect, as shown in Figure 4.16.

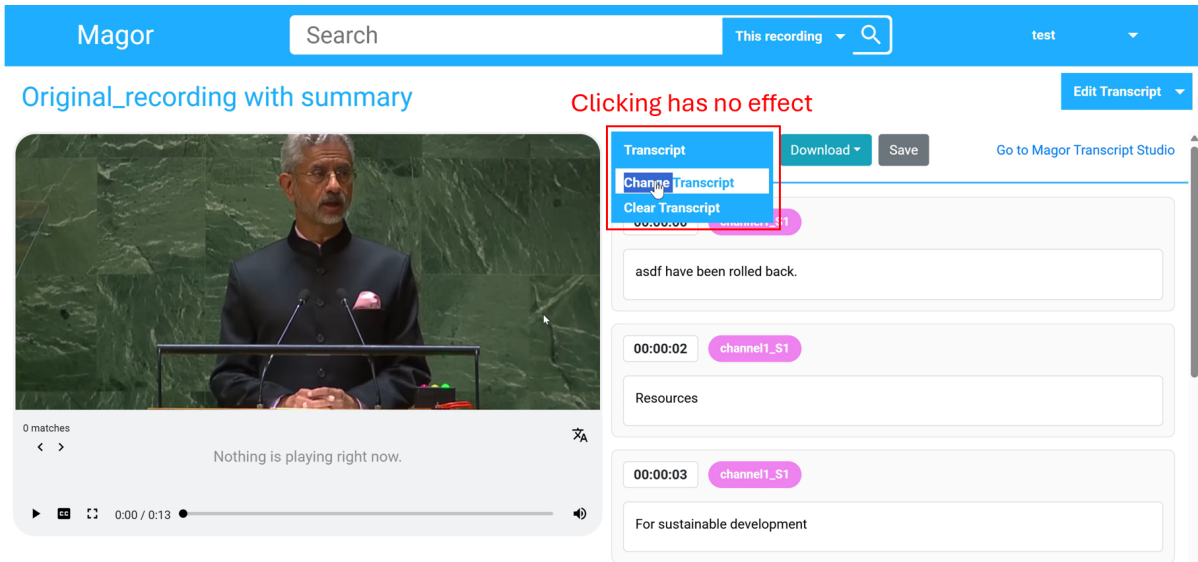


Figure 4.16: Non-functional “Transcript” drop-down in the Transcript Editor

Since users can refresh the page to clear unsaved changes, there was no need to retain the “Clear Transcript” option. The non-functional drop-down was replaced with a transcript drop-down selector listing the recording’s available transcripts, matching the selection control already provided on the Recordings page. The resulting selector is shown in Figure 4.17.

The transcript being viewed is also kept synchronized between the Recordings page and the Transcript Editor page. When a different transcript is selected, the editor updates its local sentence state with the sentences from the newly selected transcript. The Recordings page was similarly updated to track the draft file path for each transcript in component state, updating it when a draft is saved and clearing it when the transcript is published,

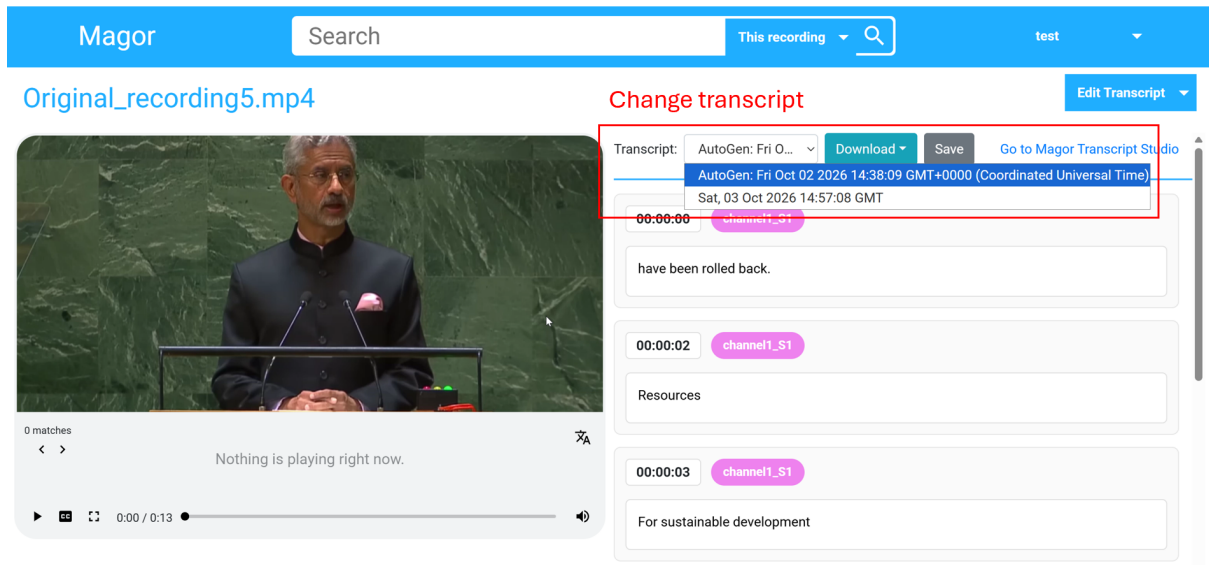


Figure 4.17: Transcript selector in the Transcript Editor following the change

since publishing removes the draft file path on the server. This allows the Recordings page to reflect changes immediately after saving or publishing without requiring a manual page refresh.

4.3.6 Recordings Page UI Improvements

One issue identified on the Recordings page was the display of a recording’s full metadata and filter controls above the tab content, such as the transcript and “Ask AI” content, resulting in limited space for viewing the transcript without scrolling, as shown in Figure 4.18.

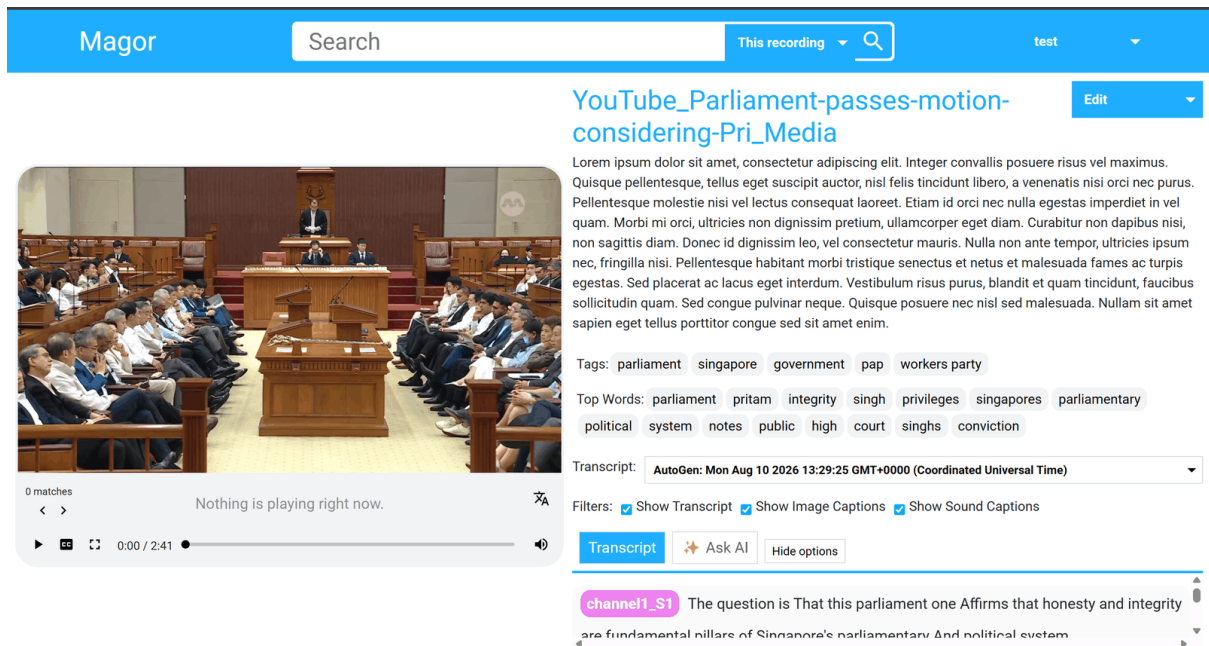


Figure 4.18: Recordings page with excessive metadata spacing

The issue is particularly evident for longer descriptions, which can occupy a substantial amount of vertical space and extend the page considerably. To address this, descriptions are now truncated to a fixed length within the heading row, with a “Show more” / “Show less” control to expand or collapse them, as shown in Figure 4.19. Any part of the description can be clicked to expand it, indicated by a highlight on hover, and clicking the “Show less” control collapses it again. This implementation effectively limits the amount of description shown and reduces the vertical space occupied by longer descriptions.

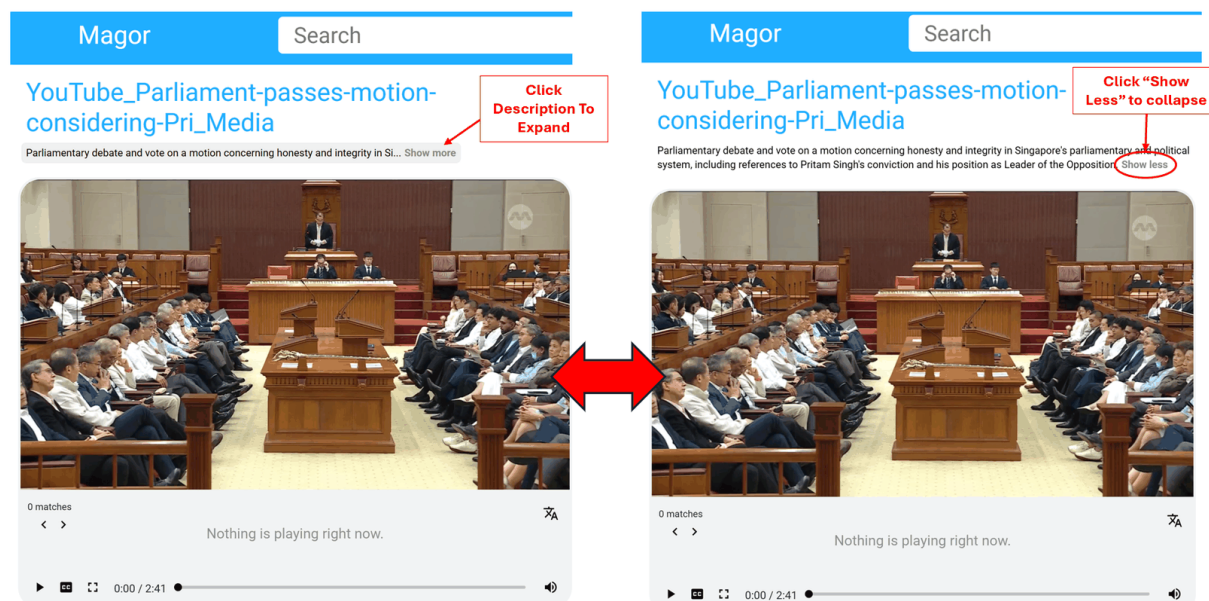


Figure 4.19: Expandable recording description

Other transcript-related controls and information, such as transcript selection, filters to show the transcript, image captions and sound captions, as well as Tags and Top Words, were consolidated into a single panel that opens and closes when the “Show options” button is clicked and is only available when the transcript tab is active, as shown in Figure 4.20. The button and the panel are rendered by separate components, so a reference to the button, `showOptionsButtonRef`, is passed from `index.js` down to `Transcript.js`, which reads the button’s on-screen position with `getBoundingClientRect()` and performs calculations to position the panel below the button. All previously existing functionality is retained, including the filters, transcript version selection, and the ability to click a tag or top word to highlight its matching occurrences and their count within the transcript.

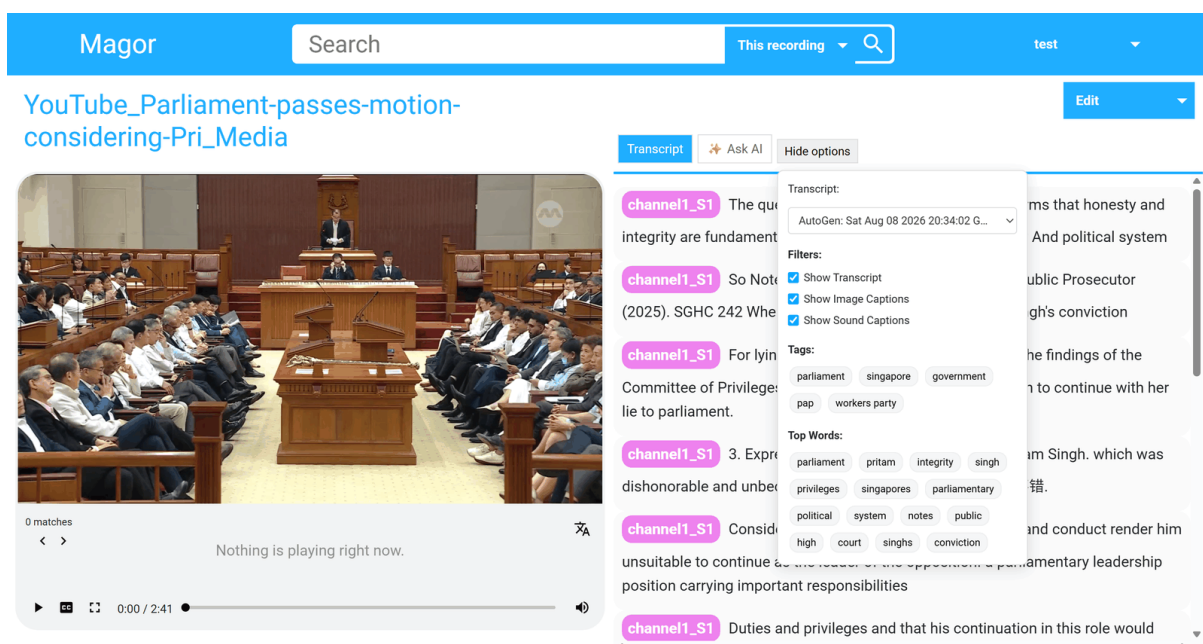


Figure 4.20: “Show options” panel opened via the dedicated button

Implementing vertical scrolling on the page was explored as an option to better utilize the available vertical space. However, testing revealed that this approach conflicts with the transcript’s automatic scrolling, which highlights and scrolls to segments currently being spoken during playback, as shown in Figure 4.21. Introducing page-level scrolling on top of this would disrupt user flow, as the page would repeatedly scroll back to the highlighted segment whenever users attempt to view another part of the page, interfering with manual scrolling.

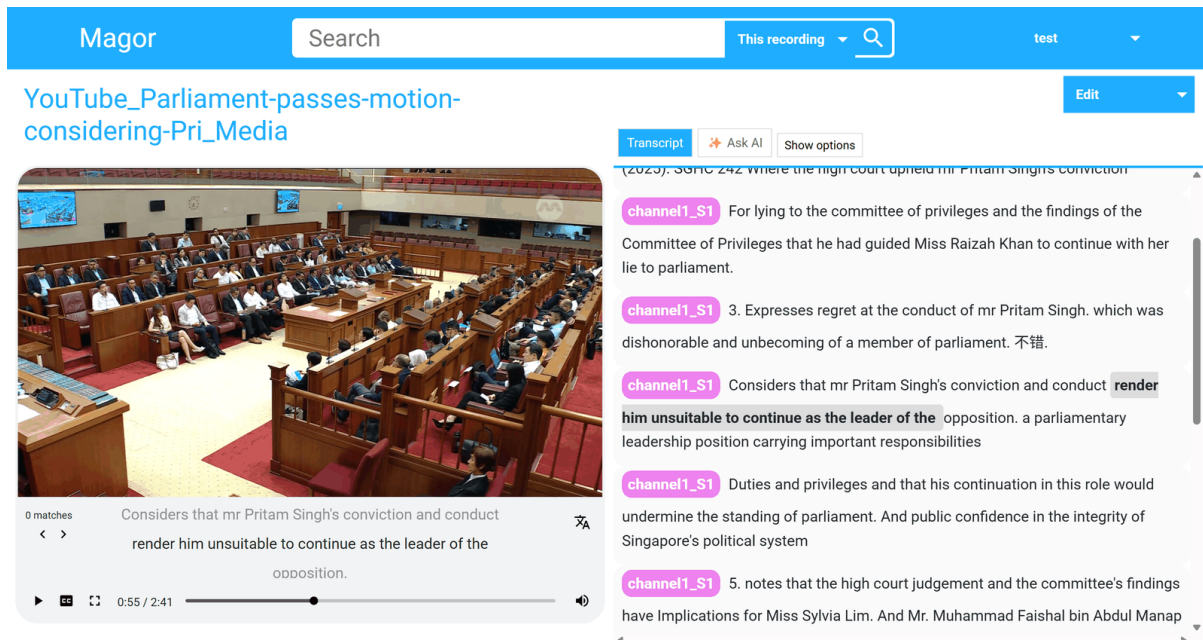


Figure 4.21: Transcript auto-scrolling during playback

With page-level scrolling found to be unsuitable, focus was shifted to improving the existing page layout, particularly the unused space above the media player. The Recordings page was restructured into four sections: a heading row with the recording's title and description on the left, alongside tab and edit controls on the right. This is followed by a split content area below the heading row with the media player on the left and the tab contents on the right, as shown in Figure 4.22.

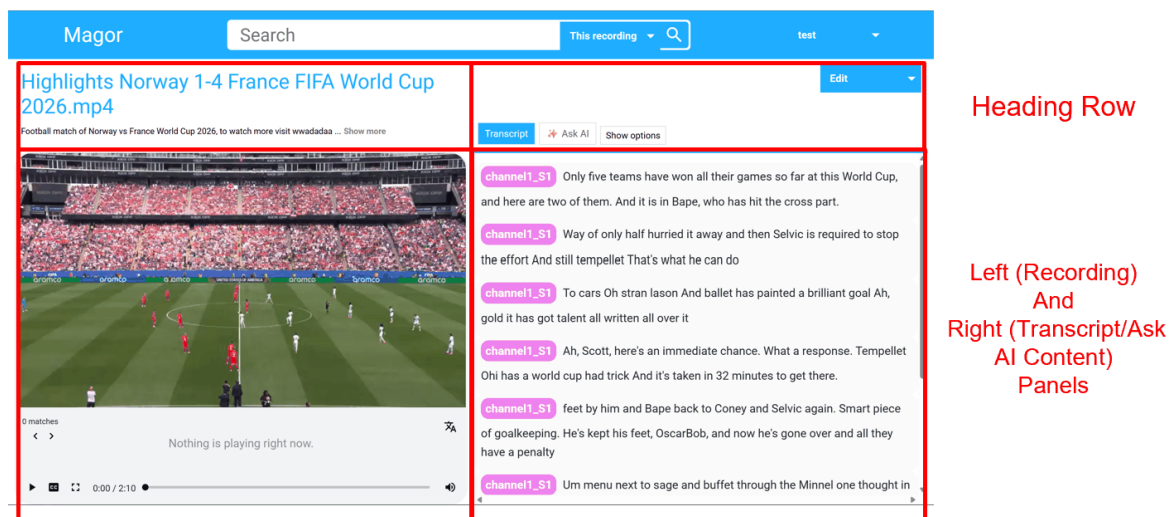


Figure 4.22: Restructured Recordings page layout

Together, these changes reduce the space occupied by the metadata and filter controls, providing a more balanced layout with greater space for the tab content without intro-

ducing page-level scrolling.

4.4 Storage and Deletion Consistency

Recordings can be deleted from two places in MAGOR: the Recordings Manager page and the Admin Panel’s Recordings page. Deleting a recording from either location initially removed only its **Media** document from the database, leaving its associated files, including transcripts, drafts, image and sound captions, and the recording’s own uploaded file, behind in storage. The linked **Transcript** documents also remained in the database, referencing a recording that no longer existed. Left unaddressed, these leftover files would accumulate indefinitely, consuming storage space for recordings that no longer exist on the platform. This section therefore documents the cascading deletion behavior implemented to address this issue, ensuring that deleting a recording also removes its associated files and database records.

4.4.1 Studio Deletion and Associated File Cleanup

Both the Recordings Manager page and the Admin Panel’s Recordings page delete recordings through the same route exposed by MAGOR Studio (`DELETE /media`), handled by `handleDeleteMediaById()` in `controller.ts` in the media routes. Although this route was already responsible for recording deletion, its original behavior did not extend beyond removing the **Media** document from the database.

The deletion behavior was therefore extended to perform the following cleanup steps in sequence:

1. All **TranscriptionRequest** documents referencing the recording are deleted.
2. The recording’s own temporary file and thumbnail are then deleted from storage.
3. Each transcript, image caption, and sound caption document linked to the recording has its associated file deleted from storage, including any saved draft version of a transcript.
4. Linked **Transcript** documents are deleted from the database.
5. Finally, the corresponding **Media** document itself is removed.

This cascading process keeps the database and storage consistent by ensuring that all resources associated with a deleted recording are removed.

4.4.2 Admin-Panel Recording Deletion Behavior

The Admin Panel’s Recordings page also required an update to ensure that recordings deleted through its interface would receive the same cascading cleanup, as described in Section 4.4.1. Before this change, deletion requests were sent to MAGOR Backend’s own recording deletion route, where the `deleteItem()` implementation removed the database records but did not delete associated files from storage. The Admin Panel was therefore updated to route recording deletions through MAGOR Studio’s `DELETE /media` endpoint instead, allowing it to use the cascading logic implemented for the Studio route. The updated routing applies to both single and bulk deletion of recordings from the Admin Panel, ensuring that the associated `TranscriptionRequest` and `Transcript` documents are removed from the database, while the associated transcripts, captions, and media files are removed from storage.

4.5 AI-Based Transcript Interaction

The “Ask AI” feature was introduced to provide users with additional ways to interact with the contents of a recording. On the Recordings page, users can access the feature by selecting the “Ask AI” tab located beside the Transcript tab. Selecting the tab replaces the transcript view with the “Ask AI” view, which contains two subsections: “Summarize” for generating a summary of the transcript and “Q&A” for asking questions about the recording’s content.

4.5.1 AI Summarization

AI summarization was introduced to provide users with a concise overview of recordings without requiring them to read through the entire transcript. This is particularly useful for longer recordings, where manually reviewing the transcript or recording to identify the main topics and key points can be laborious. The summarization feature can use the transcript as an input to generate a shorter representation of the recording’s content, including timestamps for each main point that allow users to navigate to relevant points in the recording. Figure 4.23 shows the high-level flow of this process, from submission to display.

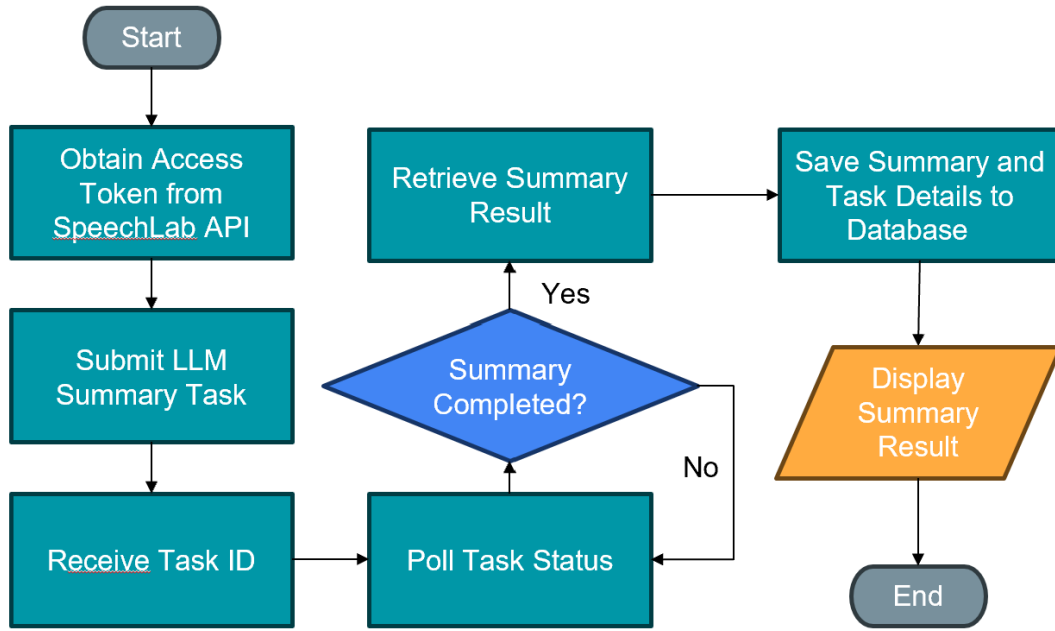


Figure 4.23: AI summarization process

The process begins with preparing the transcript for submission to the SpeechLab gateway. The `getTranscriptText()` function in `transcriptTextFormatter.js` converts the transcript into a single plain-text block. The default channel prefix produced by speaker diarization is removed from each speaker identifier before submission, so that an identifier such as `channel11_S1` is sent as `S1`. As this prefix is identical for every speaker on the same channel, it contributes no information that distinguishes one speaker from another, and removing it reduces the number of tokens consumed by the transcript. Each speech segment is represented on a separate line containing the speaker identifier, start and end timestamps, and the corresponding transcript content (`SPEAKER MM:SS.ss MM:SS.ss words`). The resulting text is submitted to the SpeechLab gateway as the `dialogue` field.

The frontend then sends the formatted transcript, the recording's media identifier, and an optional custom prompt to the `handleSummarize()` function in MAGOR Studio's `controller.ts` (AI folder). The controller first authenticates with the SpeechLab gateway, which returns an access token, before submitting a **GENERAL** task and specifying `meadow9` as the processing queue to its llm endpoint using the gateway's documented request format [15]. An example of the summarization request payload, including the formatted transcript in the `dialogue` field, is shown in Figure 4.24. The resulting access token is cached for reuse, avoiding the need to authenticate separately for each request.

```
[AI Summarize] Sending request to SpeechLab gateway:
[AI Summarize] URL: https://gateway.speechlab.sg/llm
[AI Summarize] Payload: {
  "taskType": "GENERAL",
  "queue": "meadow9",
  "dialogue": "S1 00:00.29 00:01.44 have been rolled back.\nS1 00:02.23 00:03.21 Resources\nS1 00:03.59 00:05.43 For sustainable development\nS1 00:05.65 00:07.27 Ah, severely challenged\nS1 00:08.03 00:09.27 And many countries\nS1 00:09.49 00:10.61 Really struggle\nS1 00:11.05 00:12.43 To make ends meet\nS1 00:13.51 00:14.05 Navigate",
  "userPrompt": "Summarize the following transcript in the style of YouTube's Ask AI. Begin with one <p> containing a short (2-3 sentence) overview of the conversation, ending with its supporting timestamp range in parentheses, for example (01:32-03:45). Then add one <h3> heading introducing the key points, for example <h3>Key points:</h3> (or any appropriate heading). Below it, add a single <ul> containing at most 5 <li> items, one per key point, and you must format every single point exactly like this: <li><strong>Short label:</strong> One sentence describing this point (00:52-01:03).</li>. Do not omit the <ul>/<li> tags and do not merge points into a paragraph. Focus on the key ideas, opinions, decisions and conclusions instead of individual utterances. Ignore filler words and repetitions. Never invent timestamps or information."
}
[AI Summarize] Headers: {
  Authorization: 'Bearer eyJhbGciOiJSUzI1NiIs...',
  'Content-Type': 'application/json'
}
[AI Summarize] Task created successfully. Task ID: 6abfc1d7f11a7ae7ff65820c
[AI Summarize] Media ID: 6abfc147fe91240024d828f0
[AI Summarize] Task state saved to Media document
```

Figure 4.24: Summarization request payload with the formatted transcript in the dialogue field

The SpeechLab gateway processes summarization asynchronously and initially returns a task identifier, which is stored in the recording's Media document together with a “processing” status. MAGOR Frontend then polls MAGOR Studio with a GET request (`/studioapi/api/ai/poll/:taskId`) every three seconds for up to three minutes. Automatic polling ceases when three consecutive polling requests fail, when the timeout is reached, or when the user changes tab views or navigates to another page. A manual “Check Status” control is then provided, allowing users to manually poll and check the status of the summary generation. The page interface with the manual “Check Status” control is shown in Figure 4.25.

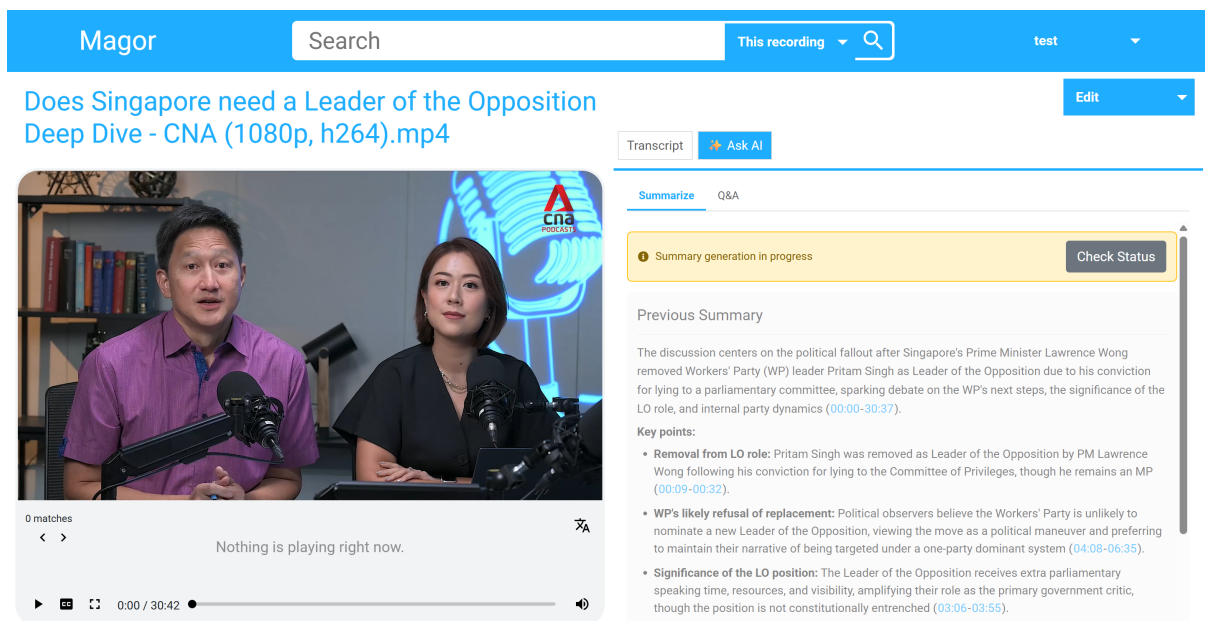


Figure 4.25: Manual summary polling interface

Once the gateway reports that the task has completed, the generated summary is stored in the database’s **Media** document and returned to the frontend for display. A generated summary can therefore be recovered whenever the summary view is reopened. The generated summary is rendered as sanitized HTML using `DOMPurify.sanitize()`, ensuring that potentially unsafe HTML content is removed before rendering. Timestamps enclosed in parentheses, matching either a “MM:SS” timestamp or “MM:SS–MM:SS” range, are identified using a regular expression and converted into clickable elements, as shown in Figure 4.26. Selecting a timestamp invokes a callback that seeks the recording’s player to the corresponding position, allowing users to move directly from a point in the generated summary to the relevant part of the recording.

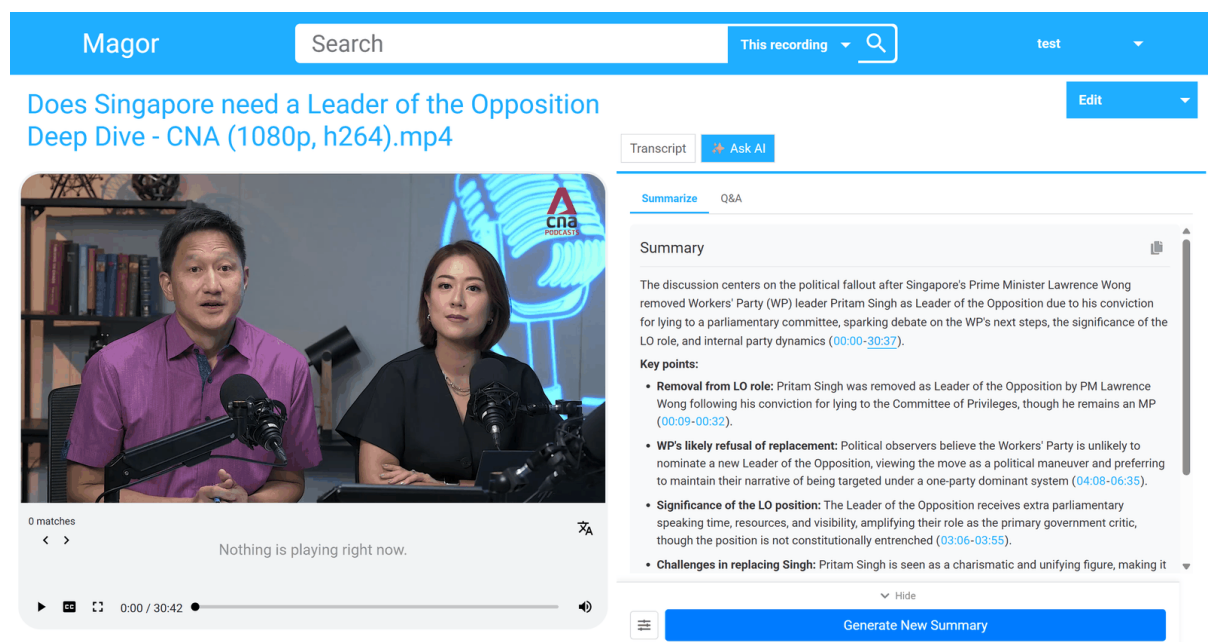


Figure 4.26: Generated transcript summary

4.5.2 Custom Summary Instructions

By default, summary generation uses a predefined prompt to produce summaries in a format similar to YouTube’s “Ask” feature. This is done by providing specific instructions to the gateway to generate a short overview paragraph followed by a “Key points” section containing up to five points, with each point consisting of a short label, a brief description, and a supporting timestamp. The default prompt is included in the summarization request as the `userPrompt` field, as shown in Figure 4.27.

```
[AI Summarize] Sending request to SpeechLab gateway:
[AI Summarize] URL: https://gateway.speechlab.sg/llm
[AI Summarize] Payload: {
  "taskType": "GENERAL",
  "queue": "meadow9",
  "dialogue": "S1 00:00.29 00:01.44 have been rolled back.\nS1 00:02.23 00:03.21 Resources\nS1 00:03.59 00:05.43 For sustainable development\nS1 00:05.65 00:07.27 Ah, severely challenged\nS1 00:08.03 00:09.27 And many countries\nS1 00:09.49 00:10.61 Really struggle\nS1 00:11.05 00:12.43 To make ends meet\nS1 00:13.51 00:14.05 Navigate",
  "userPrompt": "Summarize the following transcript in the style of YouTube's Ask AI. Begin with one <p> containing a short (2-3 sentence) overview of the conversation, ending with its supporting timestamp range in parentheses, for example (01:32-03:45). Then add one <h3> heading introducing the key points, for example <h3>Key points:</h3> (or any appropriate heading). Below it, add a single <ul> containing at most 5 <li> items, one per key point, and you must format every single point exactly like this: <li><strong>Short label:</strong> One sentence describing this point (00:52-01:03).</li>. Do not omit the <ul> and <li> tags and do not merge points into a paragraph. Focus on the key ideas, opinions, decisions and conclusions instead of individual utterances. Ignore filler words and repetitions. Never invent timestamps or information."
}
[AI Summarize] Headers: {
  Authorization: 'Bearer eyJhbGciOiJIUzI1NiIs...',
  'Content-Type': 'application/json'
}
[AI Summarize] Task created successfully. Task ID: 6abfc1d7f11a7ae7ff65820c
[AI Summarize] Media ID: 6abfc147fe91240024d828f0
[AI Summarize] Task state saved to Media document
```

Figure 4.27: Summarization request with default `userPrompt`

While this provides a predefined format for users who want a concise overview, limiting all summaries to the same structure may not suit different users or even certain types of recordings. To provide greater flexibility, users can instead select the “Custom” option and provide their own instructions for how the recording should be summarized. The supplied prompt replaces the default prompt and is submitted through the same summarization process described in Section 4.5.1. Figure 4.28 shows an example of the custom instructions entered before generating a new summary.

The screenshot shows a web interface for AI summarization. At the top, there are tabs for 'Transcript' and 'Ask AI'. Below these, there are tabs for 'Summarize' and 'Q&A'. The 'Summarize' tab is active, showing a 'Summary' section with a list of key points from a transcript. The key points are:

- Removal from LO role:** Prime Minister Lawrence Wong removed Pritam Singh as Leader of the Opposition after his conviction for lying to the Committee of Privileges, inviting the Workers' Party to nominate a replacement (00:14-00:23).
- Workers' Party response:** The Workers' Party stated it will deliberate on whether to accept the invitation to appoint a new Leader of the Opposition, with internal unity expected despite the leadership change (01:08-01:24).
- Strategic refusal likely:** Analysts Felix Tan and Eugene Tan suggest the Workers' Party is unlikely to accept the appointment to maintain their narrative of being targeted under a one-party dominant system (04:08-06:35).

Below the summary, there is a 'Summary Format' dropdown menu set to 'Custom'. A text box labeled 'Custom summary instructions:' contains the text 'Generate a summary in one concise paragraph'. At the bottom, there is a blue button labeled 'Generate New Summary'.

Figure 4.28: Custom summary input with user instructions

4.5.3 Q&A

The Q&A feature was introduced to allow users to interact with the transcript by asking questions about the recording's content. This provides a more targeted alternative to the generated summary. While summarization provides a general overview of a recording, users may instead seek specific bits of information or wish to clarify certain points without manually searching through the transcript. The Q&A feature therefore allows users to submit questions and receive answers based on the recording's transcript.

The Q&A data are stored separately in a dedicated **qas** MongoDB collection rather than being added to the **Media** document. Each document stores the associated recording using **mediaId** and stores the corresponding Q&A exchange and task information as shown in Figure 4.29. This allows a recording to store multiple Q&A exchanges while avoiding unbounded growth of the **Media** document, as a recording may accumulate an arbitrary number of questions and potentially lengthy answers over time.

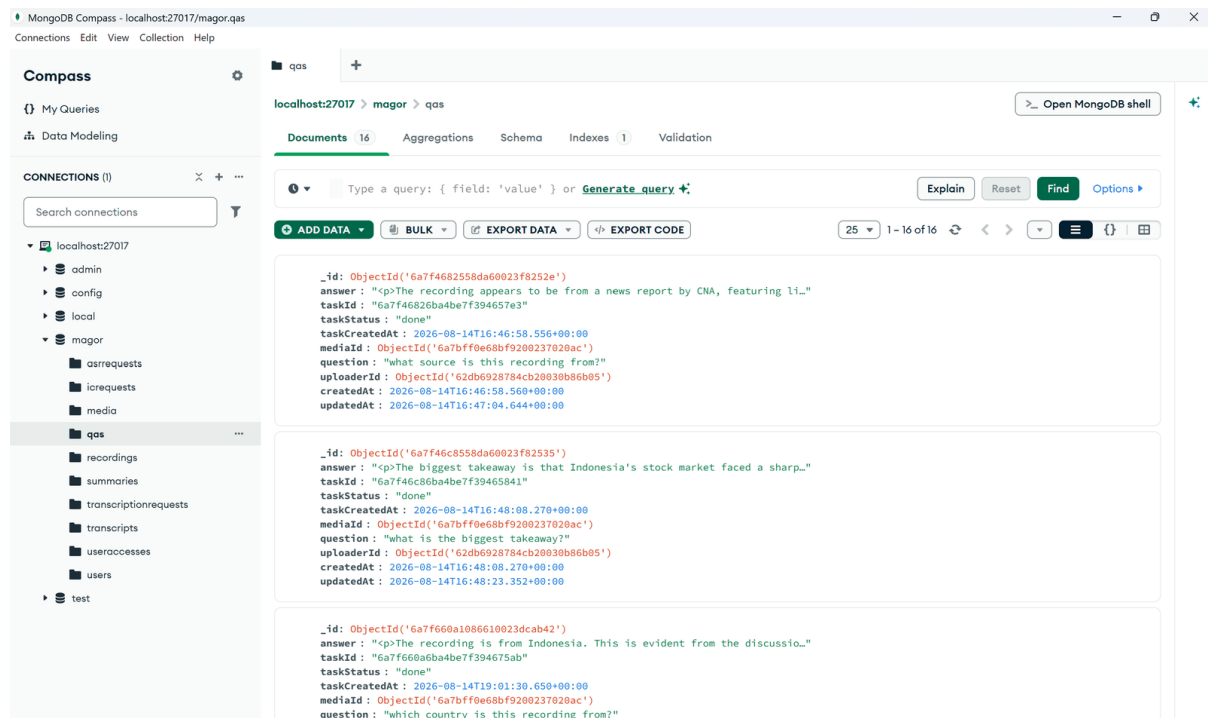


Figure 4.29: **qas** collection in MongoDB Compass

When a new question is submitted, the previous Q&A exchanges for the same recording are retrieved and converted into the **chatHistory** field for the request, allowing a follow-up question to refer back to an earlier answer. The **buildChatHistory()** function converts these exchanges into a sequence of chat messages, each labeled with the role that produced it: every past question becomes a **user** message and its corresponding answer an **assistant** message. The messages are ordered from oldest to most recent, and the new question is appended as the final **user** message.

The request is then submitted to the SpeechLab gateway as a **CHAT** task on the **meadow9** queue, with the formatted transcript supplied in the **dialogue** field and the message sequence in the **chatHistory** field, as shown in Figure 4.30. Unlike the summarization request, no **userPrompt** is supplied, as the gateway’s chat task handles the exchange directly. The complete history is resubmitted with each question rather than being retained by the gateway between requests, so no conversational session needs to be maintained on the gateway’s side.

```
2026-09-24T13:49:30.130Z - info: GET /api/ai/qa-task-status?mediaId=6aabec952567e700242aa707 304 2.002 ms - - [morgan]
[AI QA] Sending request to SpeechLab gateway:
[AI QA] URL: https://gateway.speechlab.sg/llm
[AI QA] Payload: {
  "taskType": "CHAT",
  "queue": "meadow9",
  "chatHistory": [
    {
      "role": "user",
      "content": "what is this about?"
    },
    {
      "role": "assistant",
      "content": "This transcript discusses speculation and media reports surrounding Chinese President Xi Jinping's health and conduct during his visit to New Delhi for the BRICS summit. It highlights concerns based on footage showing Xi appearing unsteady while disembarking, briefly pausing during an event, and not attending the summit's official dinner. Some sources suggest possible health issues, including a recurrence of past stroke symptoms, while others speculate about diplomatic tensions, such as perceived low reception by India or disputes involving translators, which may have led to his absence from the dinner. The report also touches on geopolitical implications, including U.S. pressure on China regarding Taiwan. Overall, it reflects international scrutiny and speculation about Xi's health and diplomatic standing."
    },
    {
      "role": "user",
      "content": "is this language chinese?"
    }
  ]
}
```

Figure 4.30: Q&A request payload showing the **chatHistory** message sequence

The remainder of the Q&A processing follows the asynchronous task and polling mechanism described in Section 4.5.1, with **handlePollQA()** providing the corresponding Q&A polling endpoint. Once completed, the generated answer is stored in the corresponding **qa** document in the database and returned to the frontend for display. Past Q&A exchanges for the recording are separately retrieved via **handleGetQAHistory()** and displayed as a running history alongside the current answer, allowing users to revisit earlier questions and answers without resubmitting them. Each past exchange is rendered as its own block, with the question labeled “Q:” and the corresponding answer labeled “A:”, to distinguish the question from its answer. Both questions and answers are rendered using the same sanitization and clickable timestamp mechanism as generated summaries. The resulting Q&A interface is shown in Figure 4.31.

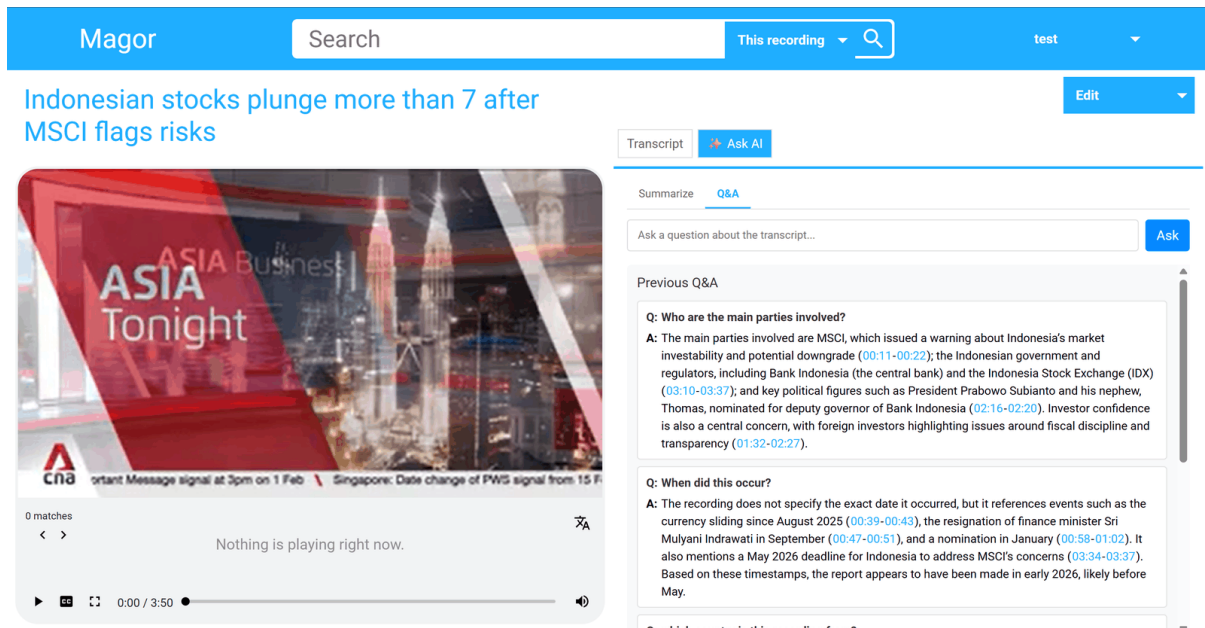


Figure 4.31: Q&A interface

As a recording may accumulate many Q&A exchanges over time, the history endpoint is paginated. Rather than using a numeric page offset, `handleGetQAHistory()` accepts an optional `before` parameter containing a timestamp and returns exchanges created before it, ordered from most recent. This avoids duplicated or skipped entries that can occur with offset-based pagination when new exchanges are added while the user is reading through the history.

Each request returns ten exchanges. To determine whether more exchanges remain without an additional query, the handler retrieves one extra document and uses its presence to set the `hasMore` flag in the response, returning only the requested ten exchanges to the client. The Q&A tab consumes this through infinite scrolling, when the history container is scrolled to its lower boundary and `hasMore` indicates that further exchanges exist, the next page is requested using the timestamp of the oldest currently loaded exchange as the cursor. Requests already in flight are tracked to prevent duplicate requests for the same page.

4.6 Dependency and Infrastructure Upgrades

Alongside the functional work, the platform's runtime and dependencies were updated. The three services had been left on unsupported Node.js releases, leaving the platform without security updates for its runtime. This section documents the migration of each service to Node.js 24, the Docker base image variants selected for each service, and the package upgrades needed to maintain compatibility with the updated runtime.

4.6.1 Node.js Runtime Upgrade

Each service specifies its Node.js version through the base image declared in its Dockerfile. MAGOR Frontend was running on Node.js 16 while MAGOR Backend and MAGOR Studio were running on Node.js 18. Node.js 16 reached end of life in September 2023 and Node.js 18 in April 2025, after which neither release receives updates, including security patches [16]. All three services were therefore migrated to Node.js 24, which entered long-term support (LTS) in October 2025 and remains supported. Table 4.2 summarizes the base image changes for each service.

Service	Previous Base Image	Current Base Image
MAGOR Backend	<code>node:18</code>	<code>node:24-trixie</code>
MAGOR Studio	<code>node:18</code>	<code>node:24-trixie</code>
MAGOR Frontend	<code>node:16-alpine</code>	<code>node:24-alpine</code>

Table 4.2: Docker base image changes for the Node.js 24 upgrade

As shown in Table 4.2, the services do not all use the same base image variant. Two official Node.js variants are used, `alpine`, which is based on Alpine Linux and uses the musl C standard library, and `trixie`, which is based on Debian and uses glibc. The alpine variant produces a smaller image, while the Debian-based variant provides a fuller set of system libraries.

This distinction is relevant to packages containing native components, which require compilation and may depend on glibc. MAGOR Frontend, which was already using the Alpine variant (`node:16-alpine` to `node:24-alpine`), installs `python3`, `make`, and `g++` to provide the required build tools. MAGOR Backend and MAGOR Studio, on the other hand, depend on `nodejieba`, which contains a native component requiring glibc 2.38. The standard Node.js 24 image provided an older glibc version, causing `nodejieba` to fail to load, while the Alpine variant uses musl instead of glibc. Both services were therefore changed to the Debian-based Trixie variant, which provides the required glibc version.

4.6.2 Dependency Upgrades

Moving to a newer runtime required corresponding package upgrades, as several dependencies were either incompatible with Node.js 24 or caused build failures. Table 4.3 lists the packages changed across the three services.

Service	Package	Previous Version	Current Version
Backend	<code>nodejieba</code>	2.4.1	3.5.8
Studio	<code>nodejieba</code>	2.6.0	3.5.8
Frontend	<code>react-scripts</code>	3.4.1	5.0.1
Frontend	<code>eslint</code>	6.8.0	8.57.0
Frontend	<code>xml-js</code> , <code>xml2js</code>	1.6.11, 0.6.2	Replaced by <code>fast-xml-parser</code> 5.11.1
Frontend	<code>mime-types</code>	2.1.26	Replaced by <code>mime</code> 3.0.0

Table 4.3: Package changes across the three services

The `nodejieba` upgrades in MAGOR Backend and MAGOR Studio were required for compatibility with Node.js 24, as native modules are compiled against a specific Node.js binary interface. Both services were therefore moved to a release supporting the newer runtime.

The frontend upgrade involved its build tooling. `react-scripts` 3 uses Webpack 4, which is incompatible with the OpenSSL version used by the newer Node.js releases. It was upgraded to version 5, which uses Webpack 5, and `eslint` was also upgraded to meet the requirements of `react-scripts` 5.

Webpack 5 also no longer automatically provides browser substitutes for Node.js core modules. The `xml-js` and `xml2js` packages used for transcript parsing relied on these modules and were therefore replaced with `fast-xml-parser`, a maintained XML parsing library that does not depend on Node.js core modules. The MIME type lookups used when previewing and uploading recordings were likewise consolidated onto the `mime` package.

Chapter 5

Conclusions and Future Works

This chapter summarizes the outcomes of the project and outlines directions for future work.

5.1 Conclusion

This project set out to restore and stabilize the MAGOR platform, which had accumulated functional regressions, outdated references, and partially completed features over several development cycles, while extending it with AI-based transcript processing.

The restoration work substantially improved the platform’s usability. Administrators can once again manage users, recordings, and transcription jobs through the Admin Panel (Section 4.1). Transcripts can now be exported in multiple formats, and the original recordings can also be downloaded after upload (Section 4.2). Reviewing a transcript is also less obstructed, as captions now track playback correctly instead of drifting out of step with the audio (Section 4.3.1), and restructuring the Recordings page recovered the space previously taken by metadata and filter controls, leaving more of the screen for the transcript itself (Section 4.3.6). Storage is also now used more efficiently, as deleting a recording now removes its associated files alongside its database records (Section 4.4).

The Ask AI feature was also added to the platform (Section 4.5), providing transcript summarization and question-and-answer capabilities. These allow users to identify the main points of a recording through summarization and ask questions about information in the transcript. Finally, all three services were migrated from end-of-life Node.js releases to Node.js 24 (Section 4.6), removing their reliance on unsupported runtimes and providing a more maintainable basis for future development.

5.2 Future Works

Several areas identified during the project could be considered for further development. The current AI implementation processes each request as an asynchronous task and polls the SpeechLab gateway until the response is fully available. The answer is thus only displayed after the task is completed, which limits the interactivity of the feature for users who want to obtain information quickly. A streaming approach could be explored instead to return partial responses as they are generated, allowing users to see the answer progressively rather than waiting for the complete response.

Further enhancements could also be made to transcript filtering and organization. The Recordings page currently provides filtering based on Tags and Top Words, but additional transcript tagging could be introduced to provide users with more ways to organize and locate relevant transcripts.

The platform could also benefit from continued dependency and infrastructure maintenance. Automated dependency upgrade tools could be introduced to keep packages and libraries up to date, while regular maintenance of the Docker configurations alongside other infrastructure components would help ensure the platform remains compatible as its dependencies and services continue to change over time.

Bibliography

- [1] C. Huang, “Magor video transcript editor,” 2022. [Online]. Available: <https://hdl.handle.net/10356/156598>.
- [2] C. W. K. Yong, “Web-based speech recognition platform (on cloud and local deployment),” 2023. [Online]. Available: <https://hdl.handle.net/10356/165918>.
- [3] R. Li, P. Liang, M. Soliman, and P. Avgeriou, “Understanding software architecture erosion: A systematic mapping study,” *Journal of Software: Evolution and Process*, vol. 34, no. 3, art. e2423, 2022, doi: 10.1002/smr.2423. [Online]. Available: <https://arxiv.org/abs/2112.10934>.
- [4] M. Alfadel, D. E. Costa, E. Shihab, and B. Adams, “On the discoverability of npm vulnerabilities in Node.js projects,” *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, art. 91, May 2023, doi: 10.1145/3571848.
- [5] A. Rao, S. Aithal, and S. Singh, “Single-document abstractive text summarization: A systematic literature review,” *ACM Computing Surveys*, vol. 57, no. 3, art. 60, Nov. 2024, doi: 10.1145/3700639.
- [6] L. Huang et al., “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *ACM Transactions on Information Systems*, vol. 43, no. 2, art. 42, Jan. 2025, doi: 10.1145/3703155.
- [7] NGINX documentation. [Online]. Available: <https://nginx.org/en/docs/>.
- [8] Redis documentation. [Online]. Available: <https://redis.io/docs/>.
- [9] Docker documentation. [Online]. Available: <https://docs.docker.com/>.
- [10] MongoDB documentation. [Online]. Available: <https://www.mongodb.com/docs/>.
- [11] Express.js documentation. [Online]. Available: <https://expressjs.com/>.
- [12] React documentation. [Online]. Available: <https://react.dev/>.

- [13] Marmelab, Marmelab/react-admin: A frontend framework for building browser-based admin applications on top of REST/GraphQL APIs. [Online]. Available: <https://github.com/marmelab/react-admin>.
- [14] Speech NTU, Speech gateway API documentation. [Online]. Available: <https://speech-ntu.gitbook.io/speech-gateway/guide-1/batch-transcribe/usage>.
- [15] SpeechLab, “LLM API documentation,” Internal technical documentation, version 3, Speech and Language Laboratory, Nanyang Technological University, 2025. Unpublished.
- [16] Node.js documentation. [Online]. Available: <https://nodejs.org/>.